

Exercise Solutions

Worked solutions to the exercises at the end of each chapter of *Python from Zero*. Each solution assumes you have run the chapter's companion notebook up to its exercises, so the data and variables it uses already exist. Programs are run from the `programs/` folder.

Chapter 1

1.1 Print a greeting with your own name in it.

```
print("Hello, my name is Sam!")
```

OUTPUT

```
Hello, my name is Sam!
```

Put your own name in place of Sam. Everything between the quotes is text, so Python prints it exactly as typed, including the exclamation mark.

1.2 Print the shelter's details on three lines.

```
print("Sunny Paws Animal Rescue")  
print("Open Tuesday to Sunday")  
print("10:00 to 17:00")
```

OUTPUT

```
Sunny Paws Animal Rescue  
Open Tuesday to Sunday  
10:00 to 17:00
```

Each `print` starts a new line, so three `print` lines give three lines of output.

1.3 Predict what `print("10 + 5")` and `print(10 + 5)` show.

```
print("10 + 5")
print(10 + 5)
```

OUTPUT

```
10 + 5
15
```

With quotes, "10 + 5" is a string, so Python prints the characters as they are. Without quotes, it is a calculation, so Python works it out first and prints the answer.

1.4 Work out the shelter's opening hours per week, then in minutes.

```
print(7 * 6)
print(7 * 6 * 60)
```

OUTPUT

```
42
2520
```

Sunny Paws is open 42 hours a week, which is 2,520 minutes. Python works through `7 * 6 * 60` from left to right, so you can multiply as many numbers as you like in one line.

1.5 Make a name tag for Peanut.

```
print("+-----+")
print("| Hello, my name is      |")
print("| PEANUT                  |")
print("| Labrador mix, age 4.5    |")
print("| Looking for a home!      |")
print("+-----+")
```

OUTPUT

```
+-----+
| Hello, my name is      |
| PEANUT                 |
| Labrador mix, age 4.5  |
| Looking for a home!    |
+-----+
```

Each string between the top and bottom borders is exactly 28 characters long, so the `|` on the right lines up. If yours is wobbly, add or remove spaces before the closing `|` until it matches the border lines.

1.6 Fix three broken lines.

```
print("Sunny Paws")
print("Maple Falls")
print("Captain")
```

OUTPUT

```
Sunny Paws
Maple Falls
Captain
```

The first line was missing its closing quote (`SyntaxError: unterminated string literal`). The second had a capital `P`, which Python does not recognize (`NameError: name 'Print' is not defined`). The third used curly quotes, which must be replaced with straight `"` quotes typed on the keyboard (`SyntaxError: invalid character`).

1.7 Add two comments to the welcome sign.

```
# Print the welcome sign for the front door of Sunny Paws.
print("=====")
print("  Welcome to Sunny Paws")
print("  Animal Rescue, Maple Falls")
print("")
print("  Open Tuesday to Sunday")
print("  10:00 to 17:00") # Closed on Mondays
print("=====")
```

OUTPUT

```
=====
  Welcome to Sunny Paws
  Animal Rescue, Maple Falls

  Open Tuesday to Sunday
  10:00 to 17:00
=====
```

The output is exactly the same as before, because Python ignores everything from a `#` to the end of the line. The comments are only for people reading the code.

1.8 Work out how much cat food 16 cats eat per day and per week.

```
print(16 * 60)
print(16 * 60 * 7)
```

OUTPUT

```
960
6720
```

Sixteen cats eating 60 g each need 960 g a day, and seven days of that is 6,720 g, a little under 7 kg a week. No parentheses are needed, because all the operations are multiplications.

Chapter 2

2.1 Use the shell to work out the fees if every kennel and cat condo held an animal adopted at full price.

```
>>> 20 * 150 + 16 * 90
4440
```

Sunny Paws would take in \$4,440. Python does the two multiplications first ($20 \times 150 = 3,000$ and $16 \times 90 = 1,440$), then adds them, so no parentheses are needed. Writing `(20 * 150) + (16 * 90)` gives the same answer and some people find it easier to read.

2.2 Compare `"Captain"` and `print("Captain")` in the shell.

```
>>> "Captain"
'Captain'
>>> print("Captain")
Captain
```

On its own, `"Captain"` is a value, and the shell shows it the way you would write it in Python, with quotes, so you can tell it is a string. `print()` shows the text the way a person should read it, without quotes.

2.3 Add a Markdown cell of notes on the three ways to run Python.

```
## My notes on running Python

- **Shell:** type one line at a time after `>>>`; nothing is saved.
- **Notebook:** code in cells with results underneath, plus notes like these.
- **Script:** a `.py` file that runs from top to bottom.
```

Change the cell type to Markdown (press **Esc**, then **M**) before typing, and run it with **Shift + Enter** to see it formatted. Your wording can be different; what matters is the heading (`##`) and the bulleted list (`-`).

2.4 Explore run order with `print(nickname)` and `nickname = "Cap"`.

Running the cells from top to bottom, the first one fails because `nickname` has not been set up yet:

```
print(nickname)
```

OUTPUT

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[1], line 1  
----> 1 print(nickname)  
  
NameError: name 'nickname' is not defined
```

Run the second cell, then go back and run the first one again, and it works:

```
nickname = "Cap"
```

```
print(nickname)
```

OUTPUT

```
Cap
```

Restart Kernel and Run All Cells makes the kernel forget `nickname`, then runs the cells from the top, so the `print` cell fails again with the same `NameError`, and JupyterLab stops there. To make the notebook work from a fresh start, move the `nickname = "Cap"` cell above the `print` cell (select it, press **Esc**, then drag it up, or cut it with **X** and paste it above with **Shift + V**).

2.5 Write and run a volunteer badge script.

```
# programs/ch02_volunteer_badge.py  
# Prints my volunteer badge for Sunny Paws.  
  
print("*****")  
print("*  Sam Rivera          *")  
print("*  Volunteer            *")  
print("*  Sunny Paws Animal Rescue *")  
print("*****")
```

```
python3 ch02_volunteer_badge.py    # macOS and Linux
py ch02_volunteer_badge.py         # Windows
```

OUTPUT

```
*****
*   Sam Rivera               *
*   Volunteer                *
*   Sunny Paws Animal Rescue *
*****
```

Use your own name, and adjust the spaces so the right-hand border lines up. Pressing F5 in IDLE shows the same badge in the IDLE Shell window. To run it from the terminal, `cd` into your `programs` folder first.

2.6 Fix `ch02_broken.py` and `ch02_typo.py`.

In `ch02_broken.py`, line 5 needs its closing parenthesis:

```
# programs/ch02_broken.py
# A welcome sign with a mistake in it, on purpose.

print("=====")
print("  Welcome to Sunny Paws")
print("  Animal Rescue, Maple Falls")
print("=====")
```

```
python3 ch02_broken.py
```

OUTPUT

```
=====
  Welcome to Sunny Paws
  Animal Rescue, Maple Falls
=====
```

In `ch02_typo.py`, line 6 should say `print`, not `prnt`:

```
# programs/ch02_typo.py
# Another sign with a mistake. This one runs until it hits the problem.

print("=====")
print("  Welcome to Sunny Paws")
print("  Animal Rescue, Maple Falls")
print("=====")
```

```
python3 ch02_typo.py
```

OUTPUT

```
=====
  Welcome to Sunny Paws
  Animal Rescue, Maple Falls
=====
```

The tracebacks pointed straight at the problems: `'('` was never closed on line 5, and `name 'prnt' is not defined`. Did you mean: `'print'?` on line 6.

2.7 Write a script that works out the cats' daily and weekly food.

```
# programs/ch02_cat_food.py
# Works out how much dry food the shelter's cats need when all
# 16 cat condos are full. Each adult cat eats 60 g a day.

print("Grams of cat food per day:")
print(16 * 60)
print("Grams of cat food per week:")
print(16 * 60 * 7)
```

```
python3 ch02_cat_food.py
```

OUTPUT

```
Grams of cat food per day:  
960  
Grams of cat food per week:  
6720
```

The comment at the top says what the script does and where the numbers come from, so nobody has to guess what 16 and 60 mean.

2.8 Explain why a script containing only `16 * 60` shows nothing.

In a script, Python works out the value of `16 * 60` but does not show it, because only `print()` puts things on the screen. (The shell and notebooks show values automatically; scripts do not.) Running the friend's script gives no output at all. The fix:

```
print(16 * 60)
```

OUTPUT

```
960
```

Chapter 3

3.1 Total and average weight of Peanut, Scout and Kiwi.

```
peanut_kg = 28.6  
scout_kg = 28.9  
kiwi_kg = 12.2  
total_kg = peanut_kg + scout_kg + kiwi_kg  
print(total_kg)  
print(round(total_kg / 3, 1))
```

OUTPUT

```
69.7  
23.2
```

The three dogs weigh 69.7 kg together, and 23.2 kg on average. Storing the total in a variable means the average line can reuse it, and the parentheses are not needed because `total_kg / 3` is already a single step.

3.2 A 6-year-old dog and a 6-year-old cat in human years.

```
age = 6
dog_years = 15 + 9 + (age - 2) * 5
cat_years = 15 + 9 + (age - 2) * 4
print("Dog: " + str(dog_years) + " human years")
print("Cat: " + str(cat_years) + " human years")
```

OUTPUT

```
Dog: 44 human years
Cat: 40 human years
```

Both rules start with `15 + 9` for the first two years; only the amount per later year differs. Change `age` and run the cell again to try other ages (2 or older).

3.3 500 minutes as hours and minutes.

```
minutes = 500
hours = minutes // 60
leftover = minutes % 60
print(str(hours) + " hours and " + str(leftover) + " minutes")
```

OUTPUT

```
8 hours and 20 minutes
```

`//` gives the number of whole hours ($8 \times 60 = 480$), and `%` gives what is left over ($500 - 480 = 20$). Both are integers, so they need `str()` before they can be joined to the text.

3.4 A name tag for Truffle built from variables.

```

name = "Truffle"
breed = "Tuxedo"
age = 1
border = "#" * 26
print(border)
print("# Name: " + name)
print("# Breed: " + breed)
print("# Age: " + str(age))
print(border)

```

OUTPUT

```

#####
# Name: Truffle
# Breed: Tuxedo
# Age: 1
#####

```

`name` and `breed` are already strings, so they join directly. `age` is an integer, so it needs `str()`. Lining up a right-hand border would mean counting spaces for every line; Chapter 4 shows how f-strings do that for you.

3.5 Predict, then check, the types of five values.

```

print(type(7 / 7))
print(type(7 // 7))
print(type("7"))
print(type(7.0))
print(type(False))

```

OUTPUT

```

<class 'float'>
<class 'int'>
<class 'str'>
<class 'float'>
<class 'bool'>

```

`/` always gives a float (here `1.0`), while `//` between integers gives an integer. `"7"` is text because of its quotes, `7.0` is a float because of its decimal point, and `False` is a Boolean.

3.6 Fix three lines that cause errors.

```
print("Weight: " + str(28.6))
print(float("28.6"))
dog_kennels = 20
print(dog_kennels)
```

OUTPUT

```
Weight: 28.6
28.6
20
```

The first line was a `TypeError` (can only concatenate `str` (not `"float"`) to `str`), fixed by converting the number with `str()`. The second was a `ValueError`, because `"28.6"` is not a whole number; `float()` converts it. The third was a `NameError` caused by a missing `s` in `dog_kennels`.

3.7 Daily food for Poppy and Scout, checked against the feeding chart.

```
grams_per_kg = 15
poppy_kg = 30.7
scout_kg = 28.9
print(round(poppy_kg * grams_per_kg))
print(round(scout_kg * grams_per_kg))
```

OUTPUT

```
460
434
```

Both match `data/feeding_chart.txt`, which lists Poppy at 460 g and Scout at 434 g. The surprise is Poppy:

```
print(30.7 * 15)
```

OUTPUT

460.5

The unrounded answer is exactly halfway, 460.5, and `round()` sends halves to the nearest even number, so it gives 460 rather than 461. This is Python's "round half to even" rule (sometimes called banker's rounding) at work.

3.8 Add three donations with `+=`, then see how many rabbit adoptions they cover.

```
total = 0
total += 500
total += 250
total += 20
print(total)
print(total // 45)
print(total % 45)
```

OUTPUT

770
17
5

The donations add up to \$770. That covers 17 rabbit adoption fees of \$45 ($17 \times 45 = 765$), with \$5 left over.

Chapter 4

4.1 Print three facts about Captain with one `print()`, using `\t` and `\n`.

```
print("Name:\tCaptain\nAge:\t12\nColor:\tinger")
```

OUTPUT

Name: Captain
Age: 12
Color: ginger

Each `\t` jumps to the next tab stop, so the values line up, and each `\n` starts a new line. One string, three lines of output.

4.2 Slice the number out of an animal ID and the month out of a date.

```
animal_id = "SP-037"
arrived = "2026-05-01"
print(animal_id[3:])
print(arrived[5:7])
```

OUTPUT

```
037
05
```

`animal_id[3:]` starts at index 3 (just after `SP-`) and runs to the end. In `"2026-05-01"` the month sits at indexes 5 and 6, so the slice is `[5:7]`, stopping before index 7.

4.3 Clean a messy name and print it with its length.

```
messy = "   jUNIPER   "
name = messy.strip().title()
print(f"{name} has {len(name)} letters")
```

OUTPUT

```
Juniper has 7 letters
```

Chaining `strip()` and `title()` removes the spaces and fixes the case in one go. Measuring the *cleaned* name gives 7; `len(messy)` would have counted the spaces too and given 12.

4.4 Pull the date, time, a yes-or-no check and a count out of a log line.

```
line = "2026-06-07 12:02 | PRIYA SHAH | asked about Pumpkin"
print(line[:10])
print(line[11:16])
print("Pumpkin" in line)
print(line.count("|"))
```

OUTPUT

```
2026-06-07
12:02
True
2
```

The date is the first 10 characters and the time is the 5 characters after the space at index 10. `in` answers the yes-or-no question and `count()` counts the separators.

4.5 Print a one-line donation summary with format specs.

```
total = 7805
count = 130
print(f"Total: ${total:,.2f} from {count} donations "
      f"(average ${total / count:.2f})")
```

OUTPUT

```
Total: $7,805.00 from 130 donations (average $60.04)
```

`:,.2f` adds the comma and two decimal places. The average is calculated right inside the braces. The two f-strings next to each other inside `print()` are joined automatically, which keeps each line of code under 80 characters.

4.6 Print the fee table with a centered title and dollar signs.

```

dog_fee = 150
cat_fee = 90
rabbit_fee = 45
guinea_pig_fee = 25

dog_text = f"${dog_fee:.2f}"
cat_text = f"${cat_fee:.2f}"
rabbit_text = f"${rabbit_fee:.2f}"
guinea_pig_text = f"${guinea_pig_fee:.2f}"

print(f"' Sunny Paws Adoption Fees ':=^30")
print(f"'Species':<20}{'Fee':>10}")
print("-" * 30)
print(f"'Dog':<20}{dog_text:>10}")
print(f"'Cat':<20}{cat_text:>10}")
print(f"'Rabbit':<20}{rabbit_text:>10}")
print(f"'Guinea pig':<20}{guinea_pig_text:>10}")

```

OUTPUT

```

== Sunny Paws Adoption Fees ==
Species                      Fee
-----
Dog                          $150.00
Cat                          $90.00
Rabbit                       $45.00
Guinea pig                   $25.00

```

Making each fee's text first ("\$150.00") and then right-aligning the whole text keeps the dollar sign snug against the number. The title is padded with = using the fill character before ^, and the spaces inside ' Sunny Paws Adoption Fees ' keep the words from touching the = signs.

4.7 Make a boxed name tag for Captain.

```

name = "Captain"
color = "Ginger"
age = 12

print("+ " + "-" * 24 + "+")
print(f"|{name.upper():^24}|")
print(f"|{color + ' cat, age ' + str(age):^24}|")
print(f"|{'Office cat':^24}|")
print("+ " + "-" * 24 + "+")

```

OUTPUT

```

+-----+
|      CAPTAIN      |
|  Ginger cat, age 12  |
|      Office cat    |
+-----+

```

The same layout as Toast's tag, at width 24. Because `age` is a number and the line is joined with `+` inside the braces, it needs `str(age)` .

4.8 Clean any line of the visitor log into `Name: reason` .

```

line = "2026-06-02 14:53| sam whitfield|returned Shadow from foster"
first_bar = line.find("|")
second_bar = line.find("|", first_bar + 1)
visitor = line[first_bar + 1:second_bar].strip().title()
reason = line[second_bar + 1:].strip()
print(f"{visitor}: {reason}")

line = "2026-06-13 11:00 | sam whitfield | asked about Toast"
first_bar = line.find("|")
second_bar = line.find("|", first_bar + 1)
visitor = line[first_bar + 1:second_bar].strip().title()
reason = line[second_bar + 1:].strip()
print(f"{visitor}: {reason}")

```

OUTPUT

```
Sam Whitfield: returned Shadow from foster  
Sam Whitfield: asked about Toast
```

Finding the two `|` characters with `find()`, instead of counting positions by hand, is what makes the same code work on both lines even though their spacing is different. Loops (Chapter 7) let you run the same code many times without copying it, and Chapter 13 uses one to clean the whole log.

Chapter 5

Save each solution that uses `input()` as a `.py` file and run it from its folder (on Windows, type `py` instead of `python3`). The transcripts show what was typed after each prompt.

5.1 Print Adoption Day's date with `sep`.

```
print("2026", "10", "03", sep="/")
```

OUTPUT

```
2026/10/03
```

Setting `sep="/"` puts a slash between the three values instead of a space.

5.2 Print one line using two `print()` calls.

```
print("Counting kibble...", end=" ")  
print("400 g")
```

OUTPUT

```
Counting kibble... 400 g
```

`end=" "` replaces the usual new line with a space, so the second `print()` continues on the same line.

5.3 Ask for a visitor and an animal, and print a tidy sentence.

```
# visitor_card.py
visitor = input("Your name: ").strip().title()
animal = input("Which animal would you like to meet? ").strip().title()
print(f"{visitor} would like to meet {animal}.")
```

```
python3 visitor_card.py
```

OUTPUT

```
Your name:   zoe martin
Which animal would you like to meet? TRUFFLE
Zoe Martin would like to meet Truffle.
```

Chaining `.strip().title()` straight onto each `input()` tidies the answers the moment they arrive, so the sentence looks right whatever the user typed.

5.4 Convert pounds to kilograms.

```
# pounds_to_kg.py
pounds = float(input("Weight in pounds: "))
kilograms = pounds * 0.4536
print(f"{pounds} pounds is about {kilograms:.1f} kg.")
```

```
python3 pounds_to_kg.py
```

OUTPUT

```
Weight in pounds: 27
27.0 pounds is about 12.2 kg.
```

The answer may have decimals, so it is converted with `float()`. The `:.1f` format spec shows one decimal place, and 27 pounds comes out at 12.2 kg as expected.

5.5 Write a cat version of the pet age converter.

```
# cat_years.py
# Cat age converter (cats aged 2 and up)

# Input
name = input("What is the cat's name? ").strip().title()
age = float(input(f"How old is {name}, in years? "))

# Process
human_years = 15 + 9 + (age - 2) * 4

# Output
print(f"{name} is about {human_years:.1f} in human years.")
```

```
python3 cat_years.py
```

OUTPUT

```
What is the cat's name? bear
How old is Bear, in years? 3
Bear is about 28.0 in human years.
```

Only the process step changes: 4 human years per year after the second instead of 5. The test with a 3-year-old cat gives 28, matching the poster. Like the dog version, it is only right for cats aged 2 or older.

5.6 Plan and write a dog version of the fee calculator.

```

# dog_fee_calculator.py
# Plan: adoption fee calculator for dogs
# Input
#   1. Ask for the adopter's name and tidy it.
#   2. Ask how many dogs (whole number).
#   3. Ask for an extra donation in dollars (0 for none).
# Process
#   4. Fees = dogs * 150. Total = fees + donation.
# Output
#   5. Print a receipt with name, dogs, fees, donation and total.

dog_fee = 150

# Input
adopter = input("Adopter's name: ").strip().title()
dogs = int(input("How many dogs are being adopted? "))
donation = float(input("Extra donation in dollars (0 for none): "))

# Process
fees = dogs * dog_fee
total = fees + donation

# Output
print()
print("=" * 32)
print(f"{'SUNNY PAWS RECEIPT':^32}")
print("=" * 32)
print(f"Adopter:          {adopter}")
print(f"Dogs adopted:     {dogs}")
print(f"Adoption fees:    ${fees:>10.2f}")
print(f"Donation:         ${donation:>10.2f}")
print("-" * 32)
print(f"Total to pay:     ${total:>10.2f}")
print("=" * 32)

```

```
python3 dog_fee_calculator.py
```

OUTPUT

```
Adopter's name: omar haddad
How many dogs are being adopted? 1
Extra donation in dollars (0 for none): 30
```

```
=====
          SUNNY PAWS RECEIPT
=====
Adopter:          Omar Haddad
Dogs adopted:     1
Adoption fees:    $    150.00
Donation:         $    30.00
-----
Total to pay:     $    180.00
=====
```

The plan, written as comments, maps directly onto the three labeled parts of the code. Only the fee, the variable names and the wording changed from the cat version.

5.7 Work out a dog's daily food and meal size.

```
# daily_food.py
# Input
name = input("Dog's name: ").strip().title()
weight = float(input(f"How much does {name} weigh, in kg? "))

# Process
grams_per_day = round(weight * 15)
grams_per_meal = grams_per_day / 2

# Output
print(f"{name} needs {grams_per_day} g of dry food a day,")
print(f"which is {grams_per_meal:.1f} g per meal.")
```

```
python3 daily_food.py
```

OUTPUT

```
Dog's name: scout
How much does Scout weigh, in kg? 28.9
Scout needs 434 g of dry food a day,
which is 217.0 g per meal.
```

28.9×15 is 433.5, which `round()` turns into 434, the same as on the shelter's feeding chart. Dividing by 2 gives the size of each of the two meals.

5.8 Challenge: count the donation jar.

```
# donation_jar.py
# Input
ones = int(input("How many $1 bills? "))
fives = int(input("How many $5 bills? "))
tens = int(input("How many $10 bills? "))
twenties = int(input("How many $20 bills? "))

# Process
ones_value = ones * 1
fives_value = fives * 5
tens_value = tens * 10
twenties_value = twenties * 20
total = ones_value + fives_value + tens_value + twenties_value

# Output
print()
print(f"{'Bill':<6}{'Count':>6}{'Value':>12}")
print("-" * 24)
print(f"{'$1':<6}{ones:>6}{ones_value:>12.2f}")
print(f"{'$5':<6}{fives:>6}{fives_value:>12.2f}")
print(f"{'$10':<6}{tens:>6}{tens_value:>12.2f}")
print(f"{'$20':<6}{twenties:>6}{twenties_value:>12.2f}")
print("-" * 24)
print(f"{'Total':<12}{total:>12,.2f}")
```

```
python3 donation_jar.py
```

OUTPUT

```
How many $1 bills? 14
How many $5 bills? 6
How many $10 bills? 3
How many $20 bills? 2
```

Bill	Count	Value

\$1	14	14.00
\$5	6	30.00
\$10	3	30.00
\$20	2	40.00

Total		114.00

Each bill's value is worked out in the process step, so the output step only arranges numbers. Text columns are left-aligned (`:<6`) and number columns right-aligned (`:>6` , `:>12.2f`), so the digits line up under each other.

Chapter 6

Save each solution that uses `input()` as a `.py` file and run it from its folder (on Windows, type `py` instead of `python3`). Where a solution was run more than once, each run is shown separately.

6.1 Predict whether each comparison is `True` or `False` .

```
print(7 >= 7)
print("Cat" == "cat")
print(3 != 3.0)
print("Ziggy" > "Bear")
print(not 0)
```

OUTPUT

```
True
False
False
True
True
```

`7 >= 7` is true because "or equal" is included. `"Cat" == "cat"` is false because case matters. `3 != 3.0` is false because Python treats the int 3 and the float 3.0 as equal. `"Ziggy" > "Bear"` is true because Z comes after B. `0` counts as false, so `not 0` is true.

6.2 Print how many kennels are free, or `Kennels full`.

```
kennels = 20

dogs_in = 18
free = kennels - dogs_in
if free > 0:
    print(f"{free} kennels free")
else:
    print("Kennels full")

dogs_in = 20
free = kennels - dogs_in
if free > 0:
    print(f"{free} kennels free")
else:
    print("Kennels full")
```

OUTPUT

```
2 kennels free
Kennels full
```

The number of free kennels is worked out first, then a simple `if / else` decides which message to print. Testing with 20 dogs checks the other branch. (Repeating the same code twice is clumsy; Chapter 10 shows how to wrap it in a function instead.)

6.3 Say whether Sunny Paws is open at a given hour.

```
# open_or_closed.py
hour = int(input("What hour is it (0 to 23)? "))
if 10 <= hour < 17:
    print("Sunny Paws is open.")
else:
    print("Sunny Paws is closed.")
```

```
python3 open_or_closed.py
```

OUTPUT

```
What hour is it (0 to 23)? 11
Sunny Paws is open.
```

OUTPUT

```
What hour is it (0 to 23)? 17
Sunny Paws is closed.
```

The chained comparison `10 <= hour < 17` includes 10 but not 17, so at 17:00 the shelter counts as closed, as required. `hour >= 10 and hour < 17` would work just as well.

6.4 Print what is on for a given day of the week.

```
# whats_on.py
day = input("What day is it? ").strip().lower()
if day == "monday":
    print("Closed today")
elif day == "tuesday":
    print("Open, and Dr. Osei is visiting")
else:
    print("Open 10:00 to 17:00")
```

```
python3 whats_on.py
```

OUTPUT

```
What day is it?  TUESDAY
Open, and Dr. Osei is visiting
```

OUTPUT

```
What day is it? Monday
Closed today
```

OUTPUT

```
What day is it? friday
Open 10:00 to 17:00
```

Tidying the answer with `.strip().lower()` means every comparison can use lowercase text, so `" TUESDAY"` and `"Monday"` are both recognized. The `else` covers the five other days without listing them.

6.5 Print a dog's daily food and number of meals.

```
# feeding_plan.py

# Input
name = input("Dog's name: ").strip().title()
weight = float(input(f"{name}'s weight in kg: "))
age = float(input(f"{name}'s age in years: "))

# Process
grams_per_day = round(weight * 15)
if age < 1:
    meals = 3
else:
    meals = 2
grams_per_meal = grams_per_day / meals

# Output
print(f"{name} needs {grams_per_day} g of dry food a day,")
print(f"in {meals} meals of {grams_per_meal:.1f} g.")
```

```
python3 feeding_plan.py
```

OUTPUT

```
Dog's name: olive
Olive's weight in kg: 6.8
Olive's age in years: 0.9
Olive needs 102 g of dry food a day,
in 3 meals of 34.0 g.
```

OUTPUT

```
Dog's name: kiwi
Kiwi's weight in kg: 12.2
Kiwi's age in years: 3
Kiwi needs 183 g of dry food a day,
in 2 meals of 91.5 g.
```

The `if / else` sets only the number of meals; everything else is the same calculation for every dog. Olive is under 1, so his 102 g is split into three meals of 34 g; Kiwi is an adult and gets two.

6.6 Fix the senior discount bug.

```
species = "Rabbit"
age = 9
if (species == "Dog" or species == "Cat") and age >= 8:
    print("Senior discount: half price")
else:
    print("No senior discount")
```

OUTPUT

```
No senior discount
```

The original condition is read as `species == "Dog" or ("Cat" and age >= 8)`, because `and` is worked out before `or`. `"Cat"` on its own counts as true, so for a 9-year-old the right-hand side is true, and the whole condition is true for any species. The fix writes both comparisons in full and adds parentheses around the `or`.

6.7 Print each species' daily food with `match`.

```
# meal_plan.py
species = input("Species: ").strip().lower()
match species:
    case "dog":
        print("15 g of dry food per kg of body weight a day")
    case "cat":
        print("60 g of dry food a day")
    case "rabbit":
        print("80 g of pellets a day, plus unlimited hay")
    case "guinea pig":
        print("40 g of pellets a day, plus hay and fresh vegetables")
    case _:
        print("Ask Rosa")
```

```
python3 meal_plan.py
```

OUTPUT

```
Species: Guinea pig
40 g of pellets a day, plus hay and fresh vegetables
```

OUTPUT

```
Species: CAT
60 g of dry food a day
```

OUTPUT

```
Species: hamster
Ask Rosa
```

Lowercasing the answer first means each `case` can use one lowercase value. `case _:` catches every species not on the list, like `else` in an `if` chain.

6.8 Challenge: decide whether today is Adoption Day.

```
# adoption_day.py
month = int(input("Month number (1 to 12): "))
day = int(input("Day of the month: "))
weekday = input("Day of the week: ").strip().lower()

is_adoption_day = month == 10 and day <= 7 and weekday == "saturday"
if is_adoption_day:
    print("It's Adoption Day! All fees are waived.")
else:
    print("Not Adoption Day today.")
```

```
python3 adoption_day.py
```

OUTPUT

```
Month number (1 to 12): 10
Day of the month: 3
Day of the week: Saturday
It's Adoption Day! All fees are waived.
```

OUTPUT

```
Month number (1 to 12): 10
Day of the month: 10
Day of the week: Saturday
Not Adoption Day today.
```

OUTPUT

```
Month number (1 to 12): 10
Day of the month: 3
Day of the week: saturday
It's Adoption Day! All fees are waived.
```

Adoption Day needs three things to be true at once, so the condition joins three comparisons with `and` . The first Saturday of a month must fall within the first seven days, which rules out October 10. Storing the answer in a well-named bool, `is_adoption_day` , makes the `if` read like English.

Chapter 7

7.1 Print the labels `Condo 1` to `Condo 16` .

```
for condo in range(1, 17):  
    print(f"Condo {condo}")
```

OUTPUT

```
Condo 1  
Condo 2  
Condo 3  
Condo 4  
Condo 5  
Condo 6  
Condo 7  
Condo 8  
Condo 9  
Condo 10  
Condo 11  
Condo 12  
Condo 13  
Condo 14  
Condo 15  
Condo 16
```

`range(1, 17)` starts at 1 and stops before 17, which gives exactly the 16 condo numbers.

7.2 Print a countdown from 10 to 1 on one line, then `Adoption Day!` .

```
for number in range(10, 0, -1):  
    print(number, end=" ")  
print()  
print("Adoption Day!")
```

OUTPUT

```
10 9 8 7 6 5 4 3 2 1  
Adoption Day!
```

A step of `-1` makes `range()` count down; it stops before 0. `end=" "` keeps the numbers on one line, and the empty `print()` finishes that line so `Adoption Day!` starts on a new one.

7.3 Count the letter `e` in a sentence, ignoring case.

```
text = "Sunny Paws Animal Rescue welcomes everyone"
count = 0
for letter in text.lower():
    if letter == "e":
        count += 1
print(f"Counted with a loop: {count}")
print(f"Checked with count(): {text.lower().count('e')}")
```

OUTPUT

```
Counted with a loop: 7
Checked with count(): 7
```

This is the counter pattern: start at 0, add 1 for every match. Lowercasing the text first means a capital `E` would be counted too. The `count()` method gives the same answer in one step, which is a good way to check your loop.

7.4 Print the days of the month on which the vet visits.

```
for day in range(2, 31, 7):
    print(f"Vet visit on day {day}")
```

OUTPUT

```
Vet visit on day 2
Vet visit on day 9
Vet visit on day 16
Vet visit on day 23
Vet visit on day 30
```

Start at 2, step by 7, and stop before 31 so that day 30 is still included.

7.5 How many weeks until a 2.0 kg puppy weighs at least 8 kg?

```

weight = 2.0
weeks = 0
while weight < 8:
    weight += 0.5
    weeks += 1
print(f"After {weeks} weeks the puppy weighs {weight} kg")

```

OUTPUT

```

After 12 weeks the puppy weighs 8.0 kg

```

We don't know in advance how many weeks it will take, so a `while` loop fits. Both `weight` and `weeks` change inside the loop, and the loop stops as soon as `weight < 8` is false. ($8.0 - 2.0 = 6.0$ kg, and $6.0 / 0.5 = 12$ weeks, so the answer checks out.)

7.6 Ask for dog weights until a blank line, then print the count, average and lightest weight.

```

count = 0
total = 0
lightest = 0
first = True

while True:
    text = input("Dog's weight in kg (blank to finish): ").strip()
    if text == "":
        break
    weight = float(text)
    count += 1
    total += weight
    if first or weight < lightest:
        lightest = weight
        first = False

if count == 0:
    print("No weights entered.")
else:
    print(f"Dogs: {count}")
    print(f"Average weight: {total / count:.1f} kg")
    print(f"Lightest: {lightest} kg")

```

OUTPUT

```
Dog's weight in kg (blank to finish): 28.6
Dog's weight in kg (blank to finish): 12.2
Dog's weight in kg (blank to finish): 6.5
Dog's weight in kg (blank to finish): 30.0
Dog's weight in kg (blank to finish):
Dogs: 4
Average weight: 19.3 kg
Lightest: 6.5 kg
```

And with no weights at all:

OUTPUT

```
Dog's weight in kg (blank to finish):
No weights entered.
```

The program combines a counter, a running total and a "smallest so far". Starting `lightest` at 0 wouldn't work, because no weight is smaller than 0. The `first` flag solves that: the first weight is always stored, and after that `lightest` only changes when a smaller weight arrives. The `if count == 0` check avoids dividing by zero.

7.7 Keep asking for an age until it is a whole number from 0 to 30.

```
while True:
    text = input("Age in whole years: ").strip()
    if not text.isdigit():
        print("Please type a whole number, like 4.")
        continue
    age = int(text)
    if age > 30:
        print("That seems too old for a shelter pet. Please check.")
        continue
    break
print(f"Age recorded: {age}")
```

OUTPUT

```
Age in whole years: three
Please type a whole number, like 4.
Age in whole years: 4.5
Please type a whole number, like 4.
Age in whole years: 45
That seems too old for a shelter pet. Please check.
Age in whole years: 7
Age recorded: 7
```

The checks happen in a safe order. `isdigit()` comes first, so `int()` is only ever called on text that really is a whole number (it also rejects "4.5" because of the dot). Only then is the number compared with 30. Each `continue` goes back to ask again, and the `break` at the bottom is reached only when both checks pass. `isdigit()` already rules out negative numbers, so 0 is the smallest possible value.

7.8 Print a chore grid with the opening days as rows and four chores as columns.

```
days = "TueWedThuFriSatSun"
print("    Walk  Feed  Clean Play")
for start in range(0, 18, 3):
    print(days[start:start + 3], end=" ")
    for chore in range(4):
        print("[ ]", end=" ")
    print()
```

OUTPUT

```
    Walk  Feed  Clean Play
Tue [ ] [ ] [ ] [ ]
Wed [ ] [ ] [ ] [ ]
Thu [ ] [ ] [ ] [ ]
Fri [ ] [ ] [ ] [ ]
Sat [ ] [ ] [ ] [ ]
Sun [ ] [ ] [ ] [ ]
```

The outer loop visits positions 0, 3, 6, 9, 12 and 15 of `days`, and the slice `days[start:start + 3]` picks out each three-letter day name. The inner loop prints four boxes on the same line, and the empty `print()` ends each row.

Chapter 8

8.1 Print the first cat, the last cat, the number of cats and the middle three.

```
cats = ["Toast", "Ollie", "Fern", "Sage", "Pepper", "Bear", "Tilly"]
print("First:", cats[0])
print("Last:", cats[-1])
print("How many:", len(cats))
print("Middle three:", cats[2:5])
```

OUTPUT

```
First: Toast
Last: Tilly
How many: 7
Middle three: ['Fern', 'Sage', 'Pepper']
```

`cats[-1]` is the last item however long the list is. The slice `cats[2:5]` starts at index 2 (the third cat) and stops before index 5, giving three cats with two on either side.

8.2 Build the adoption queue, then serve the first person.

```
queue = ["Zoe Martin", "Priya Shah"]
queue.append("Jordan Lee")
queue.insert(0, "Fatima Noor")
served = queue.pop(0)
print(f"Served: {served}")
print(f"Still waiting: {queue}")
```

OUTPUT

```
Served: Fatima Noor
Still waiting: ['Zoe Martin', 'Priya Shah', 'Jordan Lee']
```

`append()` adds to the back and `insert(0, ...)` to the front. `pop(0)` removes the first person and hands them back, so we can print who was served.

8.3 Print the lightest, heaviest and average cat weights, and the weights from heaviest to lightest.

```
cat_weights = [4.0, 3.9, 2.8, 2.8, 4.4, 5.5, 5.4]
print(f"Lightest: {min(cat_weights)} kg")
print(f"Heaviest: {max(cat_weights)} kg")
average = sum(cat_weights) / len(cat_weights)
print(f"Average: {average:.2f} kg")
print(sorted(cat_weights, reverse=True))
```

OUTPUT

```
Lightest: 2.8 kg
Heaviest: 5.5 kg
Average: 4.11 kg
[5.5, 5.4, 4.4, 4.0, 3.9, 2.8, 2.8]
```

`min()`, `max()` and `sum()` do the work. The average is `sum()` divided by `len()`, formatted with `:.2f`. `sorted(..., reverse=True)` gives a new list from heaviest to lightest without changing `cat_weights`.

8.4 Print the available dogs alphabetically as a numbered list, without changing the original list.

```
dogs = ["Peanut", "Scout", "Kiwi", "Daisy", "Milo",
        "Juniper", "Bruno", "Nugget", "Nova"]
for number, name in enumerate(sorted(dogs), start=1):
    print(f"{number}. {name}")
print(dogs[0])
```

OUTPUT

```
1. Bruno
2. Daisy
3. Juniper
4. Kiwi
5. Milo
6. Nova
7. Nugget
8. Peanut
9. Scout
Peanut
```

`sorted(dogs)` makes a sorted copy, and `enumerate(..., start=1)` numbers it from 1. The last line shows that `dogs` still starts with Peanut, so the original order is untouched. (Using `dogs.sort()` would have changed it.)

8.5 Use list comprehensions to make a list of daily food amounts and a list of the names of dogs under 15 kg.

```
waiting = ["Peanut", "Scout", "Kiwi", "Daisy", "Milo"]
weights = [28.6, 28.9, 12.2, 12.2, 10.7]

food = [round(weight * 15) for weight in weights]
print(food)

small = [name for name, weight in zip(waiting, weights) if weight < 15]
print(small)
```

OUTPUT

```
[429, 434, 183, 183, 160]
['Kiwi', 'Daisy', 'Milo']
```

The first comprehension applies the feeding rule to every weight. The second walks both lists together with `zip()`, keeps the pairs where the weight is under 15, and collects only the name from each pair.

8.6 Tidy, sort and join a list of dog names typed by the user.

```
answer = input("Dog names, separated by commas: ")
names = [name.strip().title() for name in answer.split(",")]
names.sort()
print(" & ".join(names))
```

OUTPUT

```
Dog names, separated by commas: milo, kiwi,SCOUT
Kiwi & Milo & Scout
```

Splitting on `" , "` alone copes with any spacing after the commas, and `strip().title()` cleans each piece. The comprehension does the cleaning in one line; a loop with `append()` would work just as well. Sorting before joining puts the names in alphabetical order.

8.7 Count the free kennels in each row and find Nugget.

```
kennels = [
    ["Peanut", "Scout", "", "Kiwi", "Daisy"],
    ["Milo", "", "Juniper", "", "Bruno"],
    ["", "Nugget", "", "Nova", ""],
    ["Olive", "", "", "", ""],
]

for row_number, row in enumerate(kennels, start=1):
    print(f"Row {row_number}: {row.count('')} free")

for row_number, row in enumerate(kennels, start=1):
    for position, dog in enumerate(row, start=1):
        if dog == "Nugget":
            print(f"Nugget is in row {row_number}, position {position}")
```

OUTPUT

```
Row 1: 1 free
Row 2: 2 free
Row 3: 3 free
Row 4: 4 free
Nugget is in row 3, position 2
```

`enumerate(..., start=1)` numbers the rows, and a second `enumerate()` inside numbers the positions within each row, so the answer is in the counting people use (row 3, position 2) rather than in indexes (`kennels[2][1]`). The list method `count('')` counts the empty strings in a row.

8.8 Print the adoption fee for each animal in a list of tuples.

```

animals = [
    ("Peanut", "Dog", 4.5),
    ("Ollie", "Cat", 14),
    ("Bramble", "Rabbit", 2),
    ("Juniper", "Dog", 10),
    ("Rusty", "Guinea pig", 1),
]

for name, species, age in animals:
    if species == "Dog":
        fee = 150
    elif species == "Cat":
        fee = 90
    elif species == "Rabbit":
        fee = 45
    else:
        fee = 25
    if (species == "Dog" or species == "Cat") and age >= 8:
        fee = fee / 2
    print(f"{name:<8} {species:<11} ${fee:.2f}")

```

OUTPUT

Peanut	Dog	\$150.00
Ollie	Cat	\$45.00
Bramble	Rabbit	\$45.00
Juniper	Dog	\$75.00
Rusty	Guinea pig	\$25.00

Unpacking in the `for` line gives each tuple's three values their own names, which makes the fee rules from Chapter 6 easy to read. Ollie (14) and Juniper (10) are seniors and pay half; the rabbit and guinea pig have no senior discount.

Chapter 9

9.1 Add a hamster fee, change the rabbit fee, and print every fee in alphabetical order.

```

fees = {"Dog": 150, "Cat": 90, "Rabbit": 45, "Guinea pig": 25}
fees["Hamster"] = 20
fees["Rabbit"] = 50
for species in sorted(fees):
    print(f"{species}: ${fees[species]}")

```

OUTPUT

```

Cat: $90
Dog: $150
Guinea pig: $25
Hamster: $20
Rabbit: $50

```

Assigning to a new key ("Hamster") adds an entry; assigning to an existing key ("Rabbit") replaces its value. Looping over `sorted(fees)` gives the keys in alphabetical order, and each value is looked up inside the loop.

9.2 Ask for a species and print its fee, using `get()` .

```

fees = {"Dog": 150, "Cat": 90, "Rabbit": 45, "Guinea pig": 25}
species = input("Which species? ").strip().capitalize()
fee = fees.get(species, 0)
if fee == 0:
    print(f"Sorry, Sunny Paws doesn't have any {species.lower()}s.")
else:
    print(f"The adoption fee for a {species.lower()} is ${fee}.")

```

OUTPUT

```

Which species? rabbit
The adoption fee for a rabbit is $45.

```

And with a species Sunny Paws doesn't have:

OUTPUT

```

Which species? Parrot
Sorry, Sunny Paws doesn't have any parrots.

```

`fees.get(species, 0)` never raises a `KeyError`: it gives 0 when the key is missing. That works here because no real fee is \$0. `capitalize()` turns whatever the user typed into the same form as the keys.

9.3 Count the payment methods of the first 15 donations and print the most common one.

```
text = ("online online card cash cash card cash cash cash online "
        "card cash cash cash online")
methods = text.split()

counts = {}
for method in methods:
    counts[method] = counts.get(method, 0) + 1
print(counts)

top_method = ""
top_count = 0
for method, count in counts.items():
    if count > top_count:
        top_method = method
        top_count = count
print(f"Most common: {top_method} ({top_count} of {len(methods)})")
```

OUTPUT

```
{'online': 4, 'card': 3, 'cash': 8}
Most common: cash (8 of 15)
```

The first loop is the counting pattern, `counts.get(method, 0) + 1`. The second loop is the "biggest so far" pattern from Chapter 7, run over `items()` so that we keep both the method and its count. (The long string is written as two strings side by side inside brackets; Python joins them into one.)

9.4 Make a record for Juniper and print it as a kennel card.

```

juniper = {
    "id": "SP-037",
    "name": "Juniper",
    "species": "Dog",
    "breed": "Shepherd mix",
    "sex": "F",
    "age": 10,
    "weight": 22.3,
    "status": "Available",
    "good_with_kids": True,
}
print("=" * 30)
for field, value in juniper.items():
    print(f"{field:>14}: {value}")
print("=" * 30)

```

OUTPUT

```

=====
           id: SP-037
        name: Juniper
     species: Dog
      breed: Shepherd mix
         sex: F
        age: 10
     weight: 22.3
    status: Available
good_with_kids: True
=====

```

One dictionary holds every fact about Juniper, each under a clear label. Looping over `items()` prints the card, and `>14` right-aligns each field name so the colons line up.

9.5 Print the senior dogs and cats, then the average weight of the cats.

This solution and the next use the eight-animal `animals` list from Section 9.7 (if you have since appended Nugget, run that first cell again).

```

for animal in animals:
    if animal["species"] in ("Dog", "Cat") and animal["age"] >= 8:
        print(f"{animal['name']} ({animal['species']}), {animal['age']} years")

cat_weights = [a["weight"] for a in animals if a["species"] == "Cat"]
average = sum(cat_weights) / len(cat_weights)
print(f"Average cat weight: {average:.2f} kg")

```

OUTPUT

```

Ollie (Cat), 14 years
Juniper (Dog), 10 years
Average cat weight: 3.93 kg

```

The `if` combines a membership test on a tuple with the age rule. For the average, a list comprehension collects the cats' weights, and `sum()` divided by `len()` does the rest: $(3.9 + 4.0 + 3.9) / 3 = 3.93$ kg.

9.6 Using the `animals` list from Section 9.7 (before Nugget was added), map each available animal's name to its species, then look up Bramble and Truffle.

```

species_of = {}
for animal in animals:
    if animal["status"] == "Available":
        species_of[animal["name"]] = animal["species"]
for name, species in species_of.items():
    print(f"{name}: {species}")
print("Bramble is a", species_of["Bramble"])
print("Truffle:", species_of.get("Truffle", "not available"))

```

OUTPUT

```
Peanut: Dog
Toast: Cat
Ollie: Cat
Bramble: Rabbit
Rusty: Guinea pig
Juniper: Dog
Bramble is a Rabbit
Truffle: not available
```

Building a dictionary in a loop works like building a list, but with `d[key] = value` instead of `append()`. Bramble is a key, so square brackets are safe; Truffle is on medical hold, so she isn't a key, and `get()` with a default avoids a `KeyError`.

9.7 Compare the Saturday and Sunday volunteers with sets.

```
saturday = {"Ken Adebayo", "Lucy Fairbanks", "Tomas Novak", "Aisha Bello"}
sunday = {"Lucy Fairbanks", "Grace Kim", "Aisha Bello", "Ben Carter"}

print("Both days:", sorted(saturday & sunday))
print("Saturday or Sunday:")
for name in sorted(saturday | sunday):
    print(" ", name)
print("Saturday only:", sorted(saturday - sunday))
print("People working the weekend:", len(saturday | sunday))
```

OUTPUT

```
Both days: ['Aisha Bello', 'Lucy Fairbanks']
Saturday or Sunday:
  Aisha Bello
  Ben Carter
  Grace Kim
  Ken Adebayo
  Lucy Fairbanks
  Tomas Novak
Saturday only: ['Ken Adebayo', 'Tomas Novak']
People working the weekend: 6
```

`&` gives the people in both sets, `|` everyone in either set (each person once), and `-` the people in the first set but not the second. The union has 6 people, not 8, because Lucy and Aisha work both days but are only counted once.

9.8 Build a dictionary that maps each day to the volunteers who work it.

```
volunteers = {
    "Ken Adebayo": {"days": ["Tue", "Wed", "Sat"],
                    "skills": ["dog walking", "training"]},
    "Lucy Fairbanks": {"days": ["Sat", "Sun"],
                       "skills": ["cat care", "photography"]},
    "Tomas Novak": {"days": ["Tue", "Thu", "Sat"],
                    "skills": ["small animals", "cleaning"]},
    "Mei Chen": {"days": ["Tue", "Fri"],
                 "skills": ["front desk", "photography"]},
}

by_day = {}
for name, info in volunteers.items():
    for day in info["days"]:
        if day not in by_day:
            by_day[day] = []
        by_day[day].append(name)

for day in "Tue Wed Thu Fri Sat Sun".split():
    names = by_day.get(day, [])
    if names:
        print(f"{day}: {' '.join(names)}")
    else:
        print(f"{day}: (nobody yet)")
```

OUTPUT

```
Tue: Ken Adebayo, Tomas Novak, Mei Chen
Wed: Ken Adebayo
Thu: Tomas Novak
Fri: Mei Chen
Sat: Ken Adebayo, Lucy Fairbanks, Tomas Novak
Sun: Lucy Fairbanks
```

This turns the data inside out: instead of "for each person, their days", we get "for each day, its people". The first time a day turns up, it gets an empty list; after that, each name is appended to that day's list. Printing in week order, with `get(day, [])` for safety, shows at a glance that Wednesday, Thursday, Friday and Sunday each have only one volunteer from this group.

Chapter 10

Solutions 10.5 to 10.7 assume you have already run the cells that define `human_years()`, `adoption_fee()`, `food_grams()` and the `animals` list from Section 10.5.

10.1 Write `greet_visitor(name)` and call it for Jordan and Priya.

```
def greet_visitor(name):
    print(f"Welcome to Sunny Paws, {name}!")
    print("Please sign the visitor book.")

greet_visitor("Jordan")
greet_visitor("Priya")
```

OUTPUT

```
Welcome to Sunny Paws, Jordan!
Please sign the visitor book.
Welcome to Sunny Paws, Priya!
Please sign the visitor book.
```

The function has a parameter, `name`, which takes a new value on each call. It prints rather than returns, which is fine because printing is its whole job.

10.2 Write `kg_to_lb(kg)` that returns the weight in pounds, rounded to 1 decimal place.

```
def kg_to_lb(kg):
    return round(kg * 2.20462, 1)

print(f"Peanut weighs {kg_to_lb(28.6)} lb.")
print(f"Coco weighs {kg_to_lb(3.0)} lb.")
```

OUTPUT

Peanut weighs 63.1 lb.
Coco weighs 6.6 lb.

Because `kg_to_lb()` returns its answer, the call can go straight inside an f-string.

10.3 Write `is_senior(species, age_years)` that returns `True` or `False`.

```
def is_senior(species, age_years):  
    return species in ("Dog", "Cat") and age_years >= 8  
  
print(is_senior("Dog", 10))  
print(is_senior("Cat", 3))  
print(is_senior("Rabbit", 9))
```

OUTPUT

True
False
False

A comparison like `age_years >= 8` already is `True` or `False`, so the function can return the whole condition without an `if`. The rabbit is 9 but is not a dog or cat, so it is not a senior.

10.4 Add an `adoption_day=False` keyword parameter to `adoption_fee()`.

```
def adoption_fee(species, age_years, status, adoption_day=False):
    """Return the adoption fee in dollars, or None if not adoptable.

    On Adoption Day (adoption_day=True) every fee is waived.
    """
    if status == "Medical hold":
        return None
    fees = {"Dog": 150, "Cat": 90, "Rabbit": 45, "Guinea pig": 25}
    if species not in fees:
        return None
    if adoption_day:
        return 0
    fee = fees[species]
    if species in ("Dog", "Cat") and age_years >= 8:
        fee = fee // 2
    return fee

print(adoption_fee("Dog", 4.5, "Available"))
print(adoption_fee("Dog", 4.5, "Available", adoption_day=True))
print(adoption_fee("Cat", 14, "Available", adoption_day=True))
print(adoption_fee("Dog", 0.9, "Medical hold", adoption_day=True))
```

OUTPUT

```
150
0
0
None
```

The default `False` means every existing call keeps working unchanged. The Adoption Day check comes *after* the medical-hold and species checks, so animals that cannot be adopted still get `None`.

10.5 Write `daily_food_total(animals)` that returns the total grams per day.

```
def daily_food_total(animals):
    """Return the total grams of food the animals need per day."""
    total = 0
    for animal in animals:
        total += food_grams(animal["species"], animal["age_years"],
                             animal["weight_kg"])
    return total

grams = daily_food_total(animals)
print(f"Today's food: {grams} g ({grams / 1000:.1f} kg)")
```

OUTPUT

```
Today's food: 1251 g (1.3 kg)
```

This is the running-total pattern from Chapter 7, with `food_grams()` doing the work for each animal: $429 + 80 + 460 + 80 + 60 + 102 + 40 = 1,251$ g.

10.6 Write `kennel_card(animal)` that returns a multi-line card.

```

def kennel_card(animal):
    """Return a kennel card for one animal as a multi-line string."""
    species = animal["species"]
    age = animal["age_years"]
    years = human_years(species, age)
    fee = adoption_fee(species, age, animal["status"])
    if years is None:
        years_text = "n/a"
    else:
        years_text = str(years)
    if fee is None:
        fee_text = "not available"
    else:
        fee_text = f"${fee}"
    lines = [
        f"Name:      {animal['name']}",
        f"Species:    {species}",
        f"Human years: {years_text}",
        f"Fee:        {fee_text}",
    ]
    return "\n".join(lines)

for animal in animals:
    if animal["name"] in ("Poppy", "Hazel", "Olive"):
        print(kennel_card(animal))
        print()

```

OUTPUT

```
Name:      Poppy
Species:   Dog
Human years: 64
Fee:       $75

Name:      Hazel
Species:   Rabbit
Human years: n/a
Fee:       $45

Name:      Olive
Species:   Dog
Human years: 13.5
Fee:       not available
```

The function returns a string, so the caller decides what to do with it (here, print it). Checking `is None` before using each answer turns Python's "no answer" into friendly text.

10.7 Write `weight_summary(weights)` that returns lightest, heaviest and average.

```
def weight_summary(weights):
    """Return the lightest, heaviest and average weight."""
    average = round(sum(weights) / len(weights), 1)
    return min(weights), max(weights), average

weights = [animal["weight_kg"] for animal in animals]
lightest, heaviest, average = weight_summary(weights)
print(f"Weights run from {lightest} kg to {heaviest} kg, "
      f"averaging {average} kg.")
```

OUTPUT

```
Weights run from 1.1 kg to 30.7 kg, averaging 10.8 kg.
```

`return a, b, c` returns a tuple of three values, unpacked into three variables in one line.

10.8 Rebuild the pet age converter as a program with `ask_pet()` and `main()`.

```

# programs/ch10_age_converter.py

"""Convert a pet's age to human years, using the shelter's poster."""

def human_years(species, age_years):
    """Return an animal's age in human years, using the shelter's poster.

    Works for dogs and cats. Returns None for any other species,
    because the poster has no rule for them.
    """
    if species == "Dog":
        per_later_year = 5
    elif species == "Cat":
        per_later_year = 4
    else:
        return None

    if age_years < 1:
        years = age_years * 15
    elif age_years < 2:
        years = 15 + (age_years - 1) * 9
    else:
        years = 15 + 9 + (age_years - 2) * per_later_year
    return round(years, 1)

def ask_pet():
    """Ask for a pet's name, species and age. Return all three."""
    name = input("Pet's name: ").strip()
    species = input("Species (Dog or Cat): ").strip().capitalize()
    age_years = float(input("Age in years: "))
    return name, species, age_years

def main():
    name, species, age_years = ask_pet()
    years = human_years(species, age_years)
    if years is None:
        print(f"Sorry, the poster has no rule for a {species.lower()}.")

```

```
    else:
        print(f"{name} is {years} in human years.")

if __name__ == "__main__":
    main()
```

```
python3 ch10_age_converter.py    # macOS and Linux
py ch10_age_converter.py        # Windows
```

OUTPUT

```
Pet's name: Kiwi
Species (Dog or Cat): dog
Age in years: 3
Kiwi is 29.0 in human years.
```

OUTPUT

```
Pet's name: Hazel
Species (Dog or Cat): Rabbit
Age in years: 3
Sorry, the poster has no rule for a rabbit.
```

`main()` reads like the plan: ask, convert, report. Kiwi shows as `29.0` because `float()` turned the typed `3` into `3.0`, and the arithmetic kept it a float. The file is also in the companion files as `programs/ch10_age_converter.py`.

Chapter 11

11.1 How many 2 kg bags of cat food for 30 days at 0.78 kg a day?

```
import math

daily_kg = 0.78
days = 30
bag_kg = 2
needed_kg = daily_kg * days
print(f"Food needed: {needed_kg:.1f} kg")
print("Bags to buy:", math.ceil(needed_kg / bag_kg))
```

OUTPUT

```
Food needed: 23.4 kg
Bags to buy: 12
```

23.4 kg divided by 2 kg is 11.7 bags. `math.ceil()` rounds up to 12, because 11 bags would run out before the month ends.

11.2 Shuffle the eight volunteers into a cleaning rota with `random.seed(5)`.

```
import random

volunteers = ["Ken Adebayo", "Lucy Fairbanks", "Omar Haddad", "Grace Kim",
              "Tomas Novak", "Aisha Bello", "Ben Carter", "Mei Chen"]
random.seed(5)
random.shuffle(volunteers)
print("Cleaning rota")
for number, name in enumerate(volunteers, start=1):
    print(f"{number}. {name}")
```

OUTPUT

```
Cleaning rota
1. Ben Carter
2. Grace Kim
3. Lucy Fairbanks
4. Ken Adebayo
5. Mei Chen
6. Omar Haddad
7. Aisha Bello
8. Tomas Novak
```

`shuffle()` changes the list in place, so there is nothing to assign. Because of the seed, you get exactly this order every time you run it.

11.3 Write `roll_dice()` and roll ten times with seed 11.

```
import random

def roll_dice():
    """Return the total of two six-sided dice."""
    return random.randint(1, 6) + random.randint(1, 6)

random.seed(11)
totals = []
for turn in range(10):
    totals.append(roll_dice())
print(totals)
print("Highest:", max(totals))
```

OUTPUT

```
[9, 8, 10, 4, 9, 11, 3, 7, 3, 11]
Highest: 11
```

Each call to `roll_dice()` uses two more numbers from the random sequence, and the seed makes the whole sequence repeatable. The loop variable `turn` is not used inside the loop; it only counts the ten turns.

11.4 How old is Captain in days, years and human years?

```
import sys
from datetime import date

sys.path.append("../programs")
from shelter_tools import human_years

today = date(2026, 6, 30)
born = date(2014, 5, 12)
days_old = (today - born).days
years_old = round(days_old / 365.25, 1)
print(f"Captain is {days_old} days old, about {years_old} years.")
print(f"In human years: {human_years('Cat', years_old)}")
```

OUTPUT

```
Captain is 4432 days old, about 12.1 years.
In human years: 64.4
```

Subtracting the dates gives a `timedelta`, and `.days` turns it into a number. Dividing by 365.25 allows for leap years. Run this in the companion notebook (or anywhere next to the `programs` folder) so the `sys.path` line can find `shelter_tools`.

11.5 Write `next_tuesday(day)` for Dr. Osei's visits.

```

from datetime import date, timedelta

def next_tuesday(day):
    """Return the first Tuesday after the given date."""
    day = day + timedelta(days=1)
    while day.weekday() != 1: # 1 means Tuesday
        day = day + timedelta(days=1)
    return day

for start in [date(2026, 6, 30), date(2026, 7, 3), date(2026, 7, 8)]:
    visit = next_tuesday(start)
    print(start, "->", visit.strftime("%A, %B %d"))

```

OUTPUT

```

2026-06-30 -> Tuesday, July 07
2026-07-03 -> Tuesday, July 07
2026-07-08 -> Tuesday, July 14

```

Starting from the *next* day is what makes June 30 (itself a Tuesday) give July 7 rather than June 30. In `weekday()` numbers, Monday is 0, so Tuesday is 1.

11.6 Mean, median and mode of ten donations.

```

import statistics

gifts = [150, 50, 250, 25, 10, 20, 50, 100, 200, 20]
print("Mean: ", statistics.mean(gifts))
print("Median:", statistics.median(gifts))
print("Mode: ", statistics.mode(gifts))

```

OUTPUT

```

Mean: 87.5
Median: 50.0
Mode: 50

```

The mean is \$87.50 but the median is only \$50: three large gifts (\$150, \$200 and \$250, from regular donors such as the Rotary Club and the Bakery) pull the mean up, while the median shows what a typical donor gives. 20 and 50 both appear twice; `mode()` reports 50 because it comes first in the list.

11.7 Write a module `ch11_volunteer_tools.py` and a program that imports it.

```
# programs/ch11_volunteer_tools.py
"""Small helpers for the Sunny Paws volunteer rota."""

OPEN_DAYS = ["Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]

def is_open_day(day_name):
    """Return True if the shelter is open on this day ("Mon" to "Sun")."""
    return day_name in OPEN_DAYS

if __name__ == "__main__":
    # Self-check: runs only when this file is run directly.
    print("Mon open?", is_open_day("Mon"))
    print("Sat open?", is_open_day("Sat"))
```

Running the module directly runs its self-check:

```
python3 ch11_volunteer_tools.py    # macOS and Linux
py ch11_volunteer_tools.py         # Windows
```

OUTPUT

```
Mon open? False
Sat open? True
```

```
# programs/ch11_use_volunteer_tools.py
"""Use the volunteer tools module."""
from ch11_volunteer_tools import OPEN_DAYS, is_open_day

print("Sunny Paws is open:", ", ".join(OPEN_DAYS))
for day in ["Mon", "Tue", "Sun"]:
    if is_open_day(day):
        print(f"{day}: open, volunteers needed")
    else:
        print(f"{day}: closed")
```

```
python3 ch11_use_volunteer_tools.py
```

OUTPUT

```
Sunny Paws is open: Tue, Wed, Thu, Fri, Sat, Sun
Mon: closed
Tue: open, volunteers needed
Sun: open, volunteers needed
```

When the second program imports the module, the module's `__name__` is `"ch11_volunteer_tools"`, not `"__main__"`, so the self-check does not run and only the program's own lines appear. Both files must be in the same folder.

Chapter 12

12.1 Name that error: syntax, runtime or logic?

```
print("Captain says hello"
```

OUTPUT

```
Cell In[1], line 1
    print("Captain says hello"
          ^
_IncompleteInputError: incomplete input
```

Syntax error: the closing bracket is missing. A notebook reports it as "incomplete input" (a special kind of `SyntaxError`); in a script, Python says `SyntaxError: '(' was never closed`.

```
kennels = 20 + "16"
```

OUTPUT

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[2], line 1  
----> 1 kennels = 20 + "16"  
  
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Runtime error, a `TypeError`: you cannot add a number and a string. Write `20 + 16` or `20 + int("16")`.

```
average = 20 + 50 / 2  
print(average)
```

OUTPUT

```
45.0
```

Logic error: no message, but the average of 20 and 50 is 35. Division happens first; write `(20 + 50) / 2`.

```
fees = {"Dog": 150}  
fees["dog"]
```

OUTPUT

```
-----  
KeyError                                Traceback (most recent call last)  
Cell In[4], line 2  
      1 fees = {"Dog": 150}  
----> 2 fees["dog"]  
  
KeyError: 'dog'
```

Runtime error, a `KeyError`: keys are case-sensitive, and the key is `"Dog"`.

```
for name in ["Peanut", "Scout"] print(name)
```

OUTPUT

```
Cell In[5], line 1
    for name in ["Peanut", "Scout"] print(name)
                                   ^
SyntaxError: invalid syntax
```

Syntax error: the `for` line needs a colon, and the body goes on its own indented line. The `^` points just *after* the missing colon, which is typical: look at the spot the arrow marks and just before it.

12.2 Write `safe_average(amounts)` that returns 0 for an empty list.

```
def safe_average(amounts):
    """Return the average of amounts (2 decimals), or 0 if empty."""
    try:
        return round(sum(amounts) / len(amounts), 2)
    except ZeroDivisionError:
        return 0

print(safe_average([20, 50, 100]))
print(safe_average([]))
```

OUTPUT

```
56.67
0
```

`len([])` is 0, so dividing raises `ZeroDivisionError`, which the `except` turns into a return value of 0. An `if not amounts:` check before dividing would work just as well (Chapter 8: an empty list is falsy).

12.3 Look up a fee, catching the `KeyError` for an unknown species.

```

fees = {"Dog": 150, "Cat": 90, "Rabbit": 45, "Guinea pig": 25}

species = input("Species: ").strip().capitalize()
try:
    print(f"The fee for a {species.lower()} is ${fees[species]}.")
except KeyError:
    print(f"Sorry, we don't have a fee for '{species}'.")
    print("We take:", ", ".join(fees))

```

Run twice (when you run the cell, Jupyter shows a box to type in):

OUTPUT

```

Species: cat
The fee for a cat is $90.

```

OUTPUT

```

Species: hamster
Sorry, we don't have a fee for 'Hamster'.
We take: Dog, Cat, Rabbit, Guinea pig

```

`.capitalize()` turns `cat` into `Cat` to match the keys. `", ".join(fees)` joins the dictionary's keys, because looping over a dictionary gives its keys (Chapter 9).

12.4 Write `ask_yes_no(prompt)` .

```
def ask_yes_no(prompt):
    """Ask until the user answers yes or no. Return True or False."""
    while True:
        answer = input(prompt).strip().lower()
        if answer in ("yes", "y"):
            return True
        if answer in ("no", "n"):
            return False
        print("Please answer yes or no.")

good_with_kids = ask_yes_no("Is the animal good with kids? ")
print("Saved:", good_with_kids)
```

OUTPUT

```
Is the animal good with kids? maybe
Please answer yes or no.
Is the animal good with kids? Y
Saved: True
```

No `try` is needed here, because nothing can raise an exception: the answer is compared with the allowed words. Lowercasing first means `Y`, `y`, `YES` and `yes` all work.

12.5 Write `kennel_name(kennels, number)` with kennels numbered from 1.

```
def kennel_name(kennels, number):
    """Return the name in kennel number (1 = first), or a message."""
    if number < 1:
        return "No such kennel"
    try:
        return kennels[number - 1]
    except IndexError:
        return "No such kennel"

kennels = ["Peanut", "Scout", "Kiwi"]
for number in [1, 3, 4, 0]:
    print(number, kennel_name(kennels, number))
```

OUTPUT

```
1 Peanut
3 Kiwi
4 No such kennel
0 No such kennel
```

Kennel 4 does not exist, so `kennels[3]` raises `IndexError`, which is caught. Kennel 0 is the trap in the hint: `kennels[0 - 1]` is `kennels[-1]`, which is a *valid* index (the last item), so no exception happens and you would get `Kiwi`. That is a logic error, and the `if number < 1:` check prevents it.

12.6 Write `food_for_dog(weight_kg)` that raises `ValueError` for impossible weights.

```
def food_for_dog(weight_kg):
    """Return grams of food a day for a dog. Raise ValueError if odd."""
    if weight_kg <= 0:
        raise ValueError(f"Weight must be over 0 kg, not {weight_kg}")
    if weight_kg > 90:
        raise ValueError(f"{weight_kg} kg is too heavy; check the scale")
    return round(weight_kg * 15)

for weight in [28.6, -2, 120]:
    try:
        print(f"{weight} kg: {food_for_dog(weight)} g")
    except ValueError as error:
        print(f"{weight} kg: {error}")
```

OUTPUT

```
28.6 kg: 429 g
-2 kg: Weight must be over 0 kg, not -2
120 kg: 120 kg is too heavy; check the scale
```

The function raises; the loop catches. Each message includes the bad value, so whoever reads it knows exactly what went wrong.

12.7 Find and fix the bug in `total_food()`.

Add a debug print inside the loop (the test data is the three animals from the exercise):

```
from shelter_tools import food_grams

def total_food(animals):
    total = 0
    for animal in animals:
        grams = food_grams(animal["species"], animal["age_years"],
                           animal["weight_kg"])
        print(f"DEBUG: {animal['name']} grams={grams}, total={total}")
        total += grams
    return total

animals = [
    {"name": "Peanut", "species": "Dog", "age_years": 4.5, "weight_kg": 28.6},
    {"name": "Waffles", "species": "Cat", "age_years": 0.6, "weight_kg": 1.7},
    {"name": "Hazel", "species": "Rabbit", "age_years": 3, "weight_kg": 2.5},
]
print(total_food(animals))
```

OUTPUT

```
DEBUG: Peanut grams=429, total=0
DEBUG: Waffles grams=80, total=0
DEBUG: Hazel grams=80, total=0
80
```

`grams` is right for each animal, but `total` stays 0 inside the loop. The `total += grams` line is indented at the same level as `for`, so it runs only once, after the loop, adding just the last animal's 80 g. Indent it into the loop (and remove the debug print):

```
def total_food(animals):
    total = 0
    for animal in animals:
        grams = food_grams(animal["species"], animal["age_years"],
                           animal["weight_kg"])

        total += grams
    return total

print(total_food(animals))
```

OUTPUT

589

$429 + 80 + 80 = 589$. Indentation decides which lines repeat; this is the same mistake as "returning too early" in Chapter 10, in reverse.

Chapter 13

These solutions run in the companion notebook, so the working directory is `notebooks/` and the data is at `../data/`. They assume you have already run:

```
import csv
import json
from pathlib import Path
```

13.1 Print the lines of the feeding chart for animals that need extra meals.

```
with open("../data/feeding_chart.txt", encoding="utf-8") as file:
    for line in file:
        if "three times a day" in line or "four small meals" in line:
            print(line.strip())
```

OUTPUT

Waffles: 80 g dry food, four small meals
Olive: 102 g dry food, three times a day

Looping over the file gives one line at a time, and `in` checks for either phrase. Waffles is a kitten and Olive is a puppy, so both eat more often.

13.2 Find the heaviest animal in `animals.csv`.

```
with open("../data/animals.csv", encoding="utf-8", newline="") as file:
    animals = list(csv.DictReader(file))

heaviest = animals[0]
for animal in animals:
    if float(animal["weight_kg"]) > float(heaviest["weight_kg"]):
        heaviest = animal

print(f"{heaviest['name']} weighs {heaviest['weight_kg']} kg")
```

OUTPUT

Poppy weighs 30.7 kg

This is the "find the biggest" pattern from Chapter 7, keeping the whole record rather than only the number. Both sides of the comparison need `float()`: compared as strings, `"9.5"` would beat `"30.7"` because `"9"` comes after `"3"`.

13.3 Count the animals of each species that are good with kids.

```
with open("../data/animals.csv", encoding="utf-8", newline="") as file:
    animals = list(csv.DictReader(file))

kid_friendly = {}
for animal in animals:
    if animal["good_with_kids"] == "yes":
        species = animal["species"]
        kid_friendly[species] = kid_friendly.get(species, 0) + 1

for species, count in kid_friendly.items():
    print(f"{species:<12}{count:>3}")
```

OUTPUT

Cat	11
Guinea pig	2
Rabbit	6
Dog	12

The counting pattern from Chapter 9, applied only to the rows that pass the `if`. The value in the file is the string `"yes"`, not `True`, so compare with the string.

13.4 Find the three donors who gave the most.

```
with open("../data/donations.csv", encoding="utf-8", newline="") as file:
    donations = list(csv.DictReader(file))

totals = {}
for donation in donations:
    donor = donation["donor"]
    totals[donor] = totals.get(donor, 0) + int(donation["amount"])

ranking = []
for donor, amount in totals.items():
    ranking.append((amount, donor))

for amount, donor in sorted(ranking, reverse=True)[:3]:
    print(f"{donor:<25}${amount:>6,}")
```

OUTPUT

Maple Falls Bakery	\$ 2,550
Maple Falls Rotary Club	\$ 1,400
Priya Shah	\$ 675

Putting the amount first in each tuple makes `sorted()` order by amount, and `reverse=True` puts the biggest first. The slice `[:3]` keeps the top three. The Rotary Club and the bakery, the shelter's regular larger donors, are well ahead.

13.5 Write who is on duty each day to `output/volunteer_days.txt`.

```

with open("../data/volunteers.json", encoding="utf-8") as file:
    data = json.load(file)

Path("output").mkdir(exist_ok=True)
days = ["Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
with open("output/volunteer_days.txt", "w", encoding="utf-8") as file:
    for day in days:
        names = []
        for volunteer in data["volunteers"]:
            if day in volunteer["days"]:
                names.append(volunteer["name"])
        file.write(f"{day}: {' '.join(names)}\n")

print(Path("output/volunteer_days.txt").read_text(encoding="utf-8"))

```

OUTPUT

```

Tue: Ken Adebayo, Tomas Novak, Mei Chen
Wed: Ken Adebayo, Grace Kim
Thu: Omar Haddad, Tomas Novak
Fri: Omar Haddad, Aisha Bello, Mei Chen
Sat: Ken Adebayo, Lucy Fairbanks, Tomas Novak, Aisha Bello
Sun: Lucy Fairbanks, Grace Kim, Aisha Bello, Ben Carter

```

For each opening day, the inner loop collects the names of the volunteers whose `days` list contains it. Monday is left out because the shelter is closed.

13.6 Count visits per day in the visitor log and save them as CSV.

```

with open("../data/visitor_log.txt", encoding="utf-8") as file:
    lines = file.readlines()

per_day = {}
for line in lines:
    line = line.strip()
    if line == "":
        continue
    when = line.split("|")[0].strip()
    date = when.split()[0]
    per_day[date] = per_day.get(date, 0) + 1

rows = []
for date, count in per_day.items():
    rows.append({"date": date, "visits": count})

Path("output").mkdir(exist_ok=True)
with open("output/visits_per_day.csv", "w", encoding="utf-8",
        newline="") as file:
    writer = csv.DictWriter(file, fieldnames=["date", "visits"])
    writer.writeheader()
    writer.writerows(rows)

text = Path("output/visits_per_day.csv").read_text(encoding="utf-8")
for line in text.splitlines()[1:6]:
    print(line)
print(f"... {len(rows)} days in total")

```

OUTPUT

```

date,visits
2026-06-02,1
2026-06-03,4
2026-06-04,4
2026-06-05,1
2026-06-06,1
... 24 days in total

```

Only the date matters here, so the code takes the first part of each line and the first word of that. The counts become a list of small dictionaries, which is exactly what `DictWriter` wants.

13.7 Add a "done" feature to the to-do list.

```

# programs/ch13_todo_done.py
"""The Sunny Paws to-do list, now with a way to tick tasks off."""
import json
from pathlib import Path

TODO_FILE = Path("output/todo.json")

def load_tasks():
    """Return the saved list of tasks, or an empty list if there is none."""
    try:
        with open(TODO_FILE, encoding="utf-8") as file:
            return json.load(file)
    except FileNotFoundError:
        print("No saved list yet, so starting a new one.")
        return []

def save_tasks(tasks):
    """Save the list of tasks as JSON."""
    TODO_FILE.parent.mkdir(exist_ok=True)
    with open(TODO_FILE, "w", encoding="utf-8") as file:
        json.dump(tasks, file, indent=2)

def show_tasks(tasks):
    print(f"You have {len(tasks)} task(s) on the list.")
    for number, task in enumerate(tasks, start=1):
        print(f" {number}. {task}")

def mark_done(tasks, answer):
    """Handle an answer like 'done 2' by removing task number 2."""
    try:
        number = int(answer[5:])
    except ValueError:
        print("Please type done followed by a task number, like: done 2")
        return
    if 1 <= number <= len(tasks):

```

```

        finished = tasks.pop(number - 1)
        print(f"Well done! Finished: {finished}")
    else:
        print(f"There is no task number {number}.")

def main():
    tasks = load_tasks()
    show_tasks(tasks)
    while True:
        answer = input("New task, 'done N', or Enter to finish: ").strip()
        if answer == "":
            break
        if answer.startswith("done "):
            mark_done(tasks, answer)
        else:
            tasks.append(answer)
    save_tasks(tasks)
    show_tasks(tasks)
    print(f"Saved to {TODO_FILE}")

if __name__ == "__main__":
    main()

```

Starting from the three tasks saved in the chapter, a run looks like this:

```
python3 ch13_todo_done.py
```

OUTPUT

```
You have 3 task(s) on the list.
  1. Order more cat litter
  2. Photograph Clover for the website
  3. Ask Dr. Osei about Olive's check-up
New task, 'done N', or Enter to finish: done 1
Well done! Finished: Order more cat litter
New task, 'done N', or Enter to finish: done 7
There is no task number 7.
New task, 'done N', or Enter to finish: done two
Please type done followed by a task number, like: done 2
New task, 'done N', or Enter to finish: Book Captain's vaccination
New task, 'done N', or Enter to finish:
You have 3 task(s) on the list.
  1. Photograph Clover for the website
  2. Ask Dr. Osei about Olive's check-up
  3. Book Captain's vaccination
Saved to output/todo.json
```

`answer[5:]` is everything after `"done "`. If that is not a whole number, `int()` raises `ValueError` and the function explains what to type (Chapter 12). Task numbers start at 1 for people but list indexes start at 0, hence `pop(number - 1)`.

13.8 Write `load_json(path, default)`.

```
def load_json(path, default):
    """Return the data in a JSON file, or default if the file is missing."""
    try:
        with open(path, encoding="utf-8") as file:
            return json.load(file)
    except FileNotFoundError:
        return default

data = load_json("../data/volunteers.json", {})
print(len(data["volunteers"]), "volunteers")

queue = load_json("output/no_such_file.json", [])
print(queue)
```

OUTPUT

```
8 volunteers
[]
```

Letting the caller choose the default makes the function reusable: a list for a to-do list or queue, a dictionary for settings. You could use it to shorten `load_tasks()` in the to-do program to one line.

Chapter 14

These solutions build on `programs/ch14_animals.py`. Each one either adds a method to a class in that file (shown indented, as it appears inside the class) or imports the classes. Run them from the `programs` folder, so that `import` finds the module and the data is at `../data/`.

14.1 Create Captain as a `Cat` object.

```
from ch14_animals import Cat

captain = Cat("OFFICE", "Captain", "Ginger", 12, 4.8, "Resident")
print(captain)
print(captain.human_years(), "human years")
print("Fee:", captain.adoption_fee())
```

OUTPUT

```
Captain (Cat, 12 years)
64 human years
Fee: None
```

Twelve cat years is $15 + 9 + 10 \times 4 = 64$ human years. The fee is `None` because `is_adoptable()` only accepts `"Available"` and `"Foster"`, so the office cat cannot be adopted by accident, which is exactly right.

14.2 Add a `kennel_card()` method.

```
# Add inside class Animal
def kennel_card(self):
    """Return a three-line card for the kennel door."""
    return (f"{self.animal_id} {self.name}\n"
            f"{self.species}, {self.breed}\n"
            f"{self.age_years} years, {self.weight_kg} kg")
```

```
from ch14_animals import Shelter

shelter = Shelter("Sunny Paws Animal Rescue")
shelter.load_csv("../data/animals.csv")
print(shelter.find("SP-001").kennel_card())
print()
print(shelter.find("SP-020").kennel_card())
```

OUTPUT

```
SP-001 Peanut
Dog, Labrador mix
4.5 years, 28.6 kg

SP-020 Ziggy
Rabbit, Dutch
6.0 years, 3.2 kg
```

The method returns the string rather than printing it, so the caller decides what to do with it: print it, write it to a file, or put it on a web page. Because `Dog` and `Cat` inherit from `Animal`, every animal gets the method.

14.3 Add an `age_group()` method and use it on every dog.

```
# Add inside class Animal
def age_group(self):
    if self.age_years < 1:
        return "young"
    if self.is_senior():
        return "senior"
    return "adult"
```

```

from ch14_animals import Shelter

shelter = Shelter("Sunny Paws Animal Rescue")
shelter.load_csv("../data/animals.csv")

groups = {}
for animal in shelter.animals:
    if animal.species == "Dog":
        group = animal.age_group()
        groups[group] = groups.get(group, []) + [animal.name]

for group, names in groups.items():
    print(f"{group} ({len(names)}): {' '.join(names[:6])}")

```

OUTPUT

```

adult (12): Peanut, Gus, Pickles, Scout, Kiwi, Coco
senior (6): Poppy, Marble, Sprout, Juniper, Duke, Bruno
young (2): Hopper, Olive

```

The method reuses `is_senior()` instead of repeating the "8 or older" rule. The loop groups the dog names by age group in a dictionary of lists, and the slice `[:6]` keeps each printed line short.

14.4 Add `average_age()` to `Shelter`.

```

# Add inside class Shelter
def average_age(self):
    ages = [animal.age_years for animal in self.animals]
    return round(sum(ages) / len(ages), 1)

```

```

from ch14_animals import Shelter

shelter = Shelter("Sunny Paws Animal Rescue")
shelter.load_csv("../data/animals.csv")
print(f"Average age: {shelter.average_age()} years")

```

OUTPUT

Average age: 5.1 years

The answer matches the 5.1 years you calculated from the dictionaries in Chapter 13. The ages are already floats, because `make_animal()` converted them when the objects were made.

14.5 Add `search(text)` to `Shelter`.

```
# Add inside class Shelter
def search(self, text):
    """Animals whose name or breed contains text (any case)."""
    text = text.lower()
    found = []
    for animal in self.animals:
        if text in animal.name.lower() or text in animal.breed.lower():
            found.append(animal)
    return found
```

```
from ch14_animals import Shelter

shelter = Shelter("Sunny Paws Animal Rescue")
shelter.load_csv("../data/animals.csv")
for word in ["beagle", "pe"]:
    print(f"Search for {word}:")
    for animal in shelter.search(word):
        print(f"  {animal}, {animal.breed}")
```

OUTPUT

Search for beagle:

Kiwi (Dog, 3.0 years), Beagle
Daisy (Dog, 7.5 years), Beagle
Milo (Dog, 1.3 years), Beagle
Olive (Dog, 0.9 years), Beagle

Search for pe:

Peanut (Dog, 4.5 years), Labrador mix
Hopper (Dog, 0.4 years), Greyhound
Pepper (Cat, 8.0 years), Tuxedo
Juniper (Dog, 10.0 years), Shepherd mix

Lowercasing both sides makes the search ignore case, so "beagle" finds the Beagle breed. "pe" matches Peanut, Hopper, Pepper and Juniper by name: in finds the letters anywhere in the word, not only at the start.

14.6 Write a Rabbit class with a hutch number.

```
from ch14_animals import Animal

class Rabbit(Animal):
    """A rabbit with its own hutch."""

    def __init__(self, animal_id, name, breed, age_years, weight_kg,
                  hutch, status="Available"):
        super().__init__(animal_id, name, "Rabbit", breed, age_years,
                          weight_kg, status)
        self.hutch = hutch

    def __str__(self):
        return f"{self.name} (Rabbit, hutch {self.hutch})"

ziggy = Rabbit("SP-020", "Ziggy", "Dutch", 6, 3.2, 3)
print(ziggy)
print(ziggy.adoption_fee(), ziggy.food_grams())
```

OUTPUT

```
Ziggy (Rabbit, hutch 3)
45 80
```

`super().__init__` sets up all the usual attributes, then the new line stores the hutch. `__str__` overrides the parent's version, while `adoption_fee()` and `food_grams()` are inherited unchanged.

14.7 Write a `Volunteer` class and load the volunteers from JSON.

```
import json

class Volunteer:
    def __init__(self, name, days, skills):
        self.name = name
        self.days = days
        self.skills = skills

    def __str__(self):
        return f"{self.name} ({', '.join(self.skills)})"

    def works_on(self, day):
        return day in self.days

with open("../data/volunteers.json", encoding="utf-8") as file:
    data = json.load(file)

volunteers = []
for record in data["volunteers"]:
    volunteers.append(Volunteer(record["name"], record["days"],
                                record["skills"]))

print("On duty on Sunday:")
for volunteer in volunteers:
    if volunteer.works_on("Sun"):
        print(" ", volunteer)
```

OUTPUT

On duty on Sunday:

Lucy Fairbanks (cat care, photography)
Grace Kim (front desk, social media)
Aisha Bello (cat care, fundraising)
Ben Carter (dog walking)

This is the same "file to dictionaries to objects" pattern as `make_animal()`, this time from JSON. `works_on()` is tiny, but it gives the question a clear name.

14.8 Add `to_dict()` and save the adoptable animals as JSON.

```
# Add inside class Animal
def to_dict(self):
    return {"id": self.animal_id, "name": self.name,
            "species": self.species, "age_years": self.age_years,
            "status": self.status}
```

```
import json
from pathlib import Path

from ch14_animals import Shelter

shelter = Shelter("Sunny Paws Animal Rescue")
shelter.load_csv("../data/animals.csv")
records = [animal.to_dict() for animal in shelter.adoptable()]

Path("output").mkdir(exist_ok=True)
with open("output/adoptable.json", "w", encoding="utf-8") as file:
    json.dump(records, file, indent=2)

print(f"Saved {len(records)} animals to output/adoptable.json")
print("The first record:")
for key, value in records[0].items():
    print(f"  {key}: {value}")
```

OUTPUT

```
Saved 31 animals to output/adoptable.json
The first record:
  id: SP-001
  name: Peanut
  species: Dog
  age_years: 4.5
  status: Available
```

`json.dump()` cannot save `Animal` objects directly (Chapter 13), but it can save dictionaries of strings and numbers. Turning each object into a dictionary first is the standard way round that, and Chapter 17 does the same in reverse when it loads the saved data.

Chapter 15

Solutions 15.1 and 15.2 run in the companion notebook after the chapter's cells, so `Path`, `photos` (the sample folder `output/pet_photos`) and `tidy_name()` already exist. The others are programs, run from the `programs` folder after `ch15_make_sample_photos.py` has made a fresh sample folder.

15.1 Count the files in `pet_photos`, including all subfolders, and add up their sizes.

```
count = 0
total = 0
for path in photos.rglob("*"):
    if path.is_file():
        count += 1
        total += path.stat().st_size

print(f"{count} files, {total:,} bytes in total")
```

OUTPUT

```
10 files, 366 bytes in total
```

`rglob("*")` visits every file and folder at any depth, including the month folders inside `photos/`. The `is_file()` check skips the folders themselves, which have a size of their own that is not part of the photos.

15.2 Preview the tidy name of every file in a folder.

```
def preview_names(folder):
    """Print each file's name and the name tidy_name() would give it."""
    for path in sorted(folder.iterdir()):
        if path.is_file():
            new_name = tidy_name(path)
            marker = ""
            if new_name != path.name:
                marker = " (would change)"
            print(f"{path.name:<26}{new_name}{marker}")

preview_names(Path("output/pet_photos_backup"))
```

OUTPUT

IMG_20260412_093000.jpg	2026-04-12_093000.jpg	(would change)
IMG_20260412_093512.JPG	2026-04-12_093512.jpg	(would change)
IMG_20260503_141020.jpeg	2026-05-03_141020.jpg	(would change)
IMG_20260521_160245.jpg	2026-05-21_160245.jpg	(would change)
IMG_20260603_101500.jpg	2026-06-03_101500.jpg	(would change)
Peanut at the park.jpg	peanut-at-the-park.jpg	(would change)
Truffle SLEEPING.PNG	truffle-sleeping.png	(would change)
adoption form Gus.pdf	adoption-form-gus.pdf	(would change)
mochi-vet-notes.txt	mochi-vet-notes.txt	

The backup folder still has the original messy names. Apart from the photos, the adoption form would change too (`adoption-form-gus.pdf`), because `tidy_name()` lowercases any name and replaces its spaces. Mochi's notes were already renamed by hand in Section 15.4, so they would stay as they are. A preview like this is a dry run for a rule before you write the program that uses it.

15.3 Add a `--prefix` option to `ch15_rename_photos.py` .

Three parts of the program change. `tidy_name()` takes the prefix:

```
def tidy_name(path, prefix=""):
    """Return the tidy file name for one photo."""
    stem = path.stem.lower().replace(" ", "-")
    if stem.startswith("img_") and len(stem) == 19:
        # A camera name like img_20260412_093000 becomes 2026-04-12_093000
        day = stem[4:12]
        stem = f"{day[:4]}-{day[4:6]}-{day[6:]}_{stem[13:]}"
    suffix = path.suffix.lower()
    if suffix == ".jpeg":
        suffix = ".jpg"
    if prefix and not stem.startswith(f"{prefix}-"):
        stem = f"{prefix}-{stem}"
    return stem + suffix
```

`plan_renames(folder, prefix="")` passes it on with `tidy_name(path, prefix)`, and `main()` gains the option and hands it over:

```
parser.add_argument("--prefix", default="",
                    help="text to put at the start of every new name")
```

```
plan = plan_renames(folder, args.prefix)
```

A dry run, the real run, and a second run:

```
python3 ch15_rename_photos.py output/pet_photos --prefix sunny-paws
```

OUTPUT

```
WOULD RENAME  IMG_20260412_093000.jpg -> sunny-paws-2026-04-12_093000.jpg
WOULD RENAME  IMG_20260412_093512.JPG -> sunny-paws-2026-04-12_093512.jpg
WOULD RENAME  IMG_20260503_141020.jpeg -> sunny-paws-2026-05-03_141020.jpg
WOULD RENAME  IMG_20260521_160245.jpg -> sunny-paws-2026-05-21_160245.jpg
WOULD RENAME  IMG_20260603_101500.jpg -> sunny-paws-2026-06-03_101500.jpg
WOULD RENAME  Peanut at the park.jpg -> sunny-paws-peanut-at-the-park.jpg
WOULD RENAME  Truffle SLEEPING.PNG -> sunny-paws-truffle-sleeping.png
This was a dry run. Add --apply to rename the files.
```

```
python3 ch15_rename_photos.py output/pet_photos --prefix sunny-paws --apply
python3 ch15_rename_photos.py output/pet_photos --prefix sunny-paws --apply
```

OUTPUT

```
RENAMED IMG_20260412_093000.jpg -> sunny-paws-2026-04-12_093000.jpg
RENAMED IMG_20260412_093512.JPG -> sunny-paws-2026-04-12_093512.jpg
RENAMED IMG_20260503_141020.jpeg -> sunny-paws-2026-05-03_141020.jpg
RENAMED IMG_20260521_160245.jpg -> sunny-paws-2026-05-21_160245.jpg
RENAMED IMG_20260603_101500.jpg -> sunny-paws-2026-06-03_101500.jpg
RENAMED Peanut at the park.jpg -> sunny-paws-peanut-at-the-park.jpg
RENAMED Truffle SLEEPING.PNG -> sunny-paws-truffle-sleeping.png
```

OUTPUT

```
Nothing to rename: every photo name is already tidy.
```

The default `""` is falsy (Chapter 6), so without `--prefix` nothing is added and the program behaves as before. The `startswith()` check keeps the program idempotent: without it, the second run would rename every photo again, to `sunny-paws-sunny-paws-...`

15.4 Welcome an animal from the command line, and refuse species the shelter does not take.

```
# programs/ch15_welcome.py
"""Welcome a new arrival by name and species, from the command line."""
import argparse

SPECIES = ["dog", "cat", "rabbit", "guinea pig"]

def main():
    parser = argparse.ArgumentParser(description="Welcome a new animal.")
    parser.add_argument("name", help="the animal's name")
    parser.add_argument("species", help="Dog, Cat, Rabbit or Guinea pig")
    args = parser.parse_args()

    species = args.species.lower()
    if species not in SPECIES:
        parser.error(f"sorry, Sunny Paws does not take {args.species}")
    print(f"Welcome to Sunny Paws, {args.name} the {species}!")

if __name__ == "__main__":
    main()
```

```
python3 ch15_welcome.py Biscuit Rabbit
python3 ch15_welcome.py "Mr Whiskers" cat
python3 ch15_welcome.py Spike Hedgehog
```

OUTPUT

```
Welcome to Sunny Paws, Biscuit the rabbit!
```

OUTPUT

```
Welcome to Sunny Paws, Mr Whiskers the cat!
```

OUTPUT

```
usage: ch15_welcome.py [-h] name species
ch15_welcome.py: error: sorry, Sunny Paws does not take Hedgehog
```

Comparing in lowercase accepts `Cat`, `cat` or `CAT`. For anything else, `parser.error()` prints the usage line and the message and stops, just like `argparse`'s own errors. The quotes around `"Mr Whiskers"` keep the two words together as one argument. (`add_argument` also has a `choices=[...]` setting that checks the value for you, but its error message lists every choice and is fussy about capital letters.)

15.5 Add a `--method` option to the weekly report.

`build_report()` gets a `method` parameter and filters the donations right after reading them:

```
def build_report(start, days, method=None):
    """Return the lines of the report for the given period."""
    end = start + timedelta(days=days - 1)
    donations = [row for row in read_csv(DATA / "donations.csv")
                  if in_period(row["date"], start, end)]
    if method is not None:
        donations = [row for row in donations if row["method"] == method]
```

The donations line says which method it counts:

```
label = "Donations"
if method is not None:
    label = f"Donations ({method} only)"
lines.append(f"{label}: {len(donations)}, ${total:,.} in total")
```

and `main()` adds the option and passes it on:

```
parser.add_argument("--method", choices=["cash", "card", "online"],
                    help="count only donations made this way")
```

```
lines = build_report(start, args.days, args.method)
```

```
python3 ch15_weekly_report.py 2026-04-01 --days 30 --method cash
```

OUTPUT

Sunny Paws weekly report

Wed 01 Apr 2026 to Thu 30 Apr 2026

=====

Donations (cash only): 7, \$535 in total

Biggest: \$200 from Maple Falls Bakery on 2026-04-08

Arrived: 7

SP-030 Bella (Cat)

SP-031 Rusty (Guinea pig)

SP-032 Bear (Cat)

SP-033 Sprout (Dog)

SP-034 Willow (Cat)

SP-035 Echo (Cat)

SP-036 Pepper (Cat)

Adopted: 1

SP-024 Hopper (Dog)

Visits in the visitor log: 0

An option with no `default` is `None` when it is left out, so `method is not None` tells the two cases apart. Everything after the filter (the total and the biggest gift) automatically works on the filtered list.

15.6 Make a dated backup that never overwrites.

```

# programs/ch15_backup.py
"""Make a dated backup copy of a folder in output/backups/.

Run it from the programs folder, for example:
    python3 ch15_backup.py output/pet_photos
"""
import argparse
import shutil
from datetime import date
from pathlib import Path

BACKUPS = Path("output/backups")

def main():
    parser = argparse.ArgumentParser(description="Back up a folder.")
    parser.add_argument("folder", help="the folder to back up")
    args = parser.parse_args()

    folder = Path(args.folder)
    if not folder.is_dir():
        print(f"There is no folder called {folder}")
        return
    target = BACKUPS / f"{folder.name}_{date.today()}"
    if target.exists():
        print(f"{target} already exists, so nothing was copied.")
        return
    BACKUPS.mkdir(parents=True, exist_ok=True)
    shutil.copytree(folder, target)
    print(f"Backed up {folder} to {target}")

if __name__ == "__main__":
    main()

```

```

python3 ch15_backup.py output/pet_photos
python3 ch15_backup.py output/pet_photos

```

OUTPUT

Backed up output/pet_photos to output/backups/pet_photos_2026-10-06

OUTPUT

output/backups/pet_photos_2026-10-06 already exists, so nothing was copied.

`date.today()` inside the f-string gives the date as `2026-10-06`, so each day's backup gets its own folder. `parents=True` creates `output/backups` the first time. The second run on the same day finds the folder already there and refuses, following the "never overwrite" habit. (Your date will be different.)

15.7 Find the newest and oldest file in a folder.

```

# programs/ch15_file_ages.py
"""Show the newest and oldest file in a folder."""
import argparse
from datetime import datetime
from pathlib import Path

def main():
    parser = argparse.ArgumentParser(
        description="Show the newest and oldest file in a folder.")
    parser.add_argument("folder", help="the folder to look in")
    args = parser.parse_args()

    files = [path for path in Path(args.folder).iterdir() if path.is_file()]
    if not files:
        print("There are no files in that folder.")
        return
    newest = files[0]
    oldest = files[0]
    for path in files:
        if path.stat().st_mtime > newest.stat().st_mtime:
            newest = path
        if path.stat().st_mtime < oldest.stat().st_mtime:
            oldest = path

    for label, path in [("Newest", newest), ("Oldest", oldest)]:
        when = datetime.fromtimestamp(path.stat().st_mtime)
        print(f"{label}: {path.name} ({when.strftime('%Y-%m-%d %H:%M:%S')})")

if __name__ == "__main__":
    main()

```

```
python3 ch15_file_ages.py ../data
```

OUTPUT

```
Newest: animals.csv (2026-10-06 14:48:50)
Oldest: volunteers.json (2026-10-06 14:48:49)
```

This is the "find the biggest" pattern from Chapter 7 run twice at once, comparing `st_mtime` values. The dates you see are when the files were saved on your computer, so yours will differ.

Chapter 16

The test files below are saved in `programs/ch16_tests/`, next to `shelter_rules.py` and `donations_report.py`, and run from that folder with `python3 -m pytest -q` followed by the file name (`py -m pytest -q ...` on Windows). `-q` keeps the output short.

16.1 Check `food_grams()` for Peanut and a kitten with `assert`, then make one check fail.

In the companion notebook, after running the cell that defines the shelter's rules:

```
assert food_grams("Dog", 4.5, 28.6) == 429
assert food_grams("Cat", 0.5, 1.2) == 80
print("Both checks passed")
```

OUTPUT

```
Both checks passed
```

Now change the expected value for Peanut in a new cell:

```
assert food_grams("Dog", 4.5, 28.6) == 430, "Peanut should get 430 g"
```

OUTPUT

```
-----
AssertionError                                Traceback (most recent call last)
Cell In[3], line 1
----> 1 assert food_grams("Dog", 4.5, 28.6) == 430, "Peanut should get 430 g"

AssertionError: Peanut should get 430 g
```

The first two checks pass silently. The last one expects the wrong value (28.6×15 is 429, not 430), so Python stops with an `AssertionError` that shows your message. Your cell number will differ.

16.2 Test that a 1-year-old dog is 15 human years and a 2-year-old cat is 24.

```
# programs/ch16_tests/test_ex16_2.py
from shelter_rules import human_years

def test_one_year_old_dog():
    assert human_years("Dog", 1) == 15

def test_two_year_old_cat():
    assert human_years("Cat", 2) == 24
```

OUTPUT

```
..
2 passed in 0.01s
```

[100%]

Ages 1 and 2 are exactly where the poster's rule changes (15 for the first year, then 9 more), so they are the places an off-by-one bug would show up.

16.3 Test that `food_grams()` rejects a hamster and that an old guinea pig pays the full fee.

```
# programs/ch16_tests/test_ex16_3.py
import pytest

from shelter_rules import adoption_fee, food_grams

def test_food_for_unknown_species_is_an_error():
    with pytest.raises(ValueError):
        food_grams("Hamster", 1, 0.1)

def test_old_guinea_pig_pays_full_fee():
    assert adoption_fee("Guinea pig", 10, "Available") == 25
```

OUTPUT

```
..  
2 passed in 0.01s
```

[100%]

`pytest.raises(ValueError)` passes only if the code inside the `with` block raises a `ValueError`. The second test makes sure the senior discount stays limited to dogs and cats.

16.4 Write `is_senior(species, age_years)` with at least three tests.

```
# programs/ch16_tests/test_ex16_4.py  
def is_senior(species, age_years):  
    """Return True for dogs and cats aged 8 or older."""  
    return species in ("Dog", "Cat") and age_years >= 8  
  
def test_ten_year_old_dog_is_senior():  
    assert is_senior("Dog", 10)  
  
def test_exactly_eight_is_senior():  
    assert is_senior("Cat", 8)  
  
def test_seven_and_a_half_is_not_senior():  
    assert not is_senior("Dog", 7.5)  
  
def test_rabbits_are_never_senior():  
    assert not is_senior("Rabbit", 9)
```

OUTPUT

```
....  
4 passed in 0.01s
```

[100%]

The function returns the result of the comparison directly. For a small exercise it is fine to keep the function and its tests in one file; in a real program the function would live in a module such as `shelter_rules.py`. `assert not ...` checks that something is false.

16.5 Write tests for `total_by_method()` first, watch them fail, then write the function.

The tests, written before the function exists:

```
# programs/ch16_tests/test_ex16_5.py
GIFTS = [
    {"date": "2026-03-01", "donor": "A", "amount": 20, "method": "cash"},
    {"date": "2026-03-02", "donor": "B", "amount": 50, "method": "online"},
    {"date": "2026-03-03", "donor": "C", "amount": 10, "method": "cash"},
]

def test_total_by_method():
    assert total_by_method(GIFTS) == {"cash": 30, "online": 50}

def test_no_gifts():
    assert total_by_method([]) == {}
```

Running them with `python3 -m pytest -q --tb=no test_ex16_5.py` (`--tb=no` hides the tracebacks and keeps only the summary):

```
OUTPUT
FF                                                                    [100%]
===== short test summary info =====
FAILED test_ex16_5.py::test_total_by_method - NameError: name 'total_by_metho...
FAILED test_ex16_5.py::test_no_gifts - NameError: name 'total_by_method' is n...
2 failed in 0.01s
```

Both fail with `NameError` because the function does not exist yet. Now add the function at the top of the file:

```
def total_by_method(donations):
    """Return a dict of totals per payment method."""
    totals = {}
    for gift in donations:
        method = gift["method"]
        totals[method] = totals.get(method, 0) + gift["amount"]
    return totals
```

OUTPUT

```
.. [100%]
2 passed in 0.01s
```

This is the same counting pattern that fixed `total_by_month() : get(method, 0)` starts each method at 0, and the gift is added to it.

16.6 Find the planted bug in `count_available()` with a failing test, then fix it.

```
# programs/ch16_tests/test_ex16_6.py
ANIMALS = [
    {"name": "Peanut", "status": "Available"},
    {"name": "Truffle", "status": "Medical hold"},
    {"name": "Scout", "status": "Available"},
]

def count_available(animals):
    count = 0
    for animal in animals:
        if animal["status"] == "available":
            count += 1
    return count

def test_count_available():
    count = count_available(ANIMALS)
    assert count == 2
```

OUTPUT

```
F [100%]
===== FAILURES =====
_____ test_count_available _____

    def test_count_available():
        count = count_available(ANIMALS)
>         assert count == 2
E         assert 0 == 2

test_ex16_6.py:19: AssertionError
===== short test summary info =====
FAILED test_ex16_6.py::test_count_available - assert 0 == 2
1 failed in 0.01s
```

The count is 0, not 2: nothing matches. The status in the data is "Available" with a capital A, and string comparison is case-sensitive (Chapter 6). The fix is one character:

```
if animal["status"] == "Available":
```

OUTPUT

```
. [100%]
1 passed in 0.01s
```

16.7 Step through the buggy `top_donor()` with a debugger, then fix it and test it.

With a `breakpoint()` added as the first line inside the loop and two gifts, a `pdb` session looks like this (we typed everything after `(Pdb)`):

```
# programs/ch16_tests/ex7.py
GIFTS = [
    {"date": "2026-01-03", "donor": "Chris Doyle", "amount": 50},
    {"date": "2026-01-10", "donor": "Maple Falls Bakery", "amount": 100},
]

def top_donor(donations):
    """Return (donor, amount) for the single biggest gift."""
    best_donor = ""
    best_amount = 0
    for gift in donations:
        breakpoint()
        if gift["amount"] > best_amount:
            best_amount = gift["amount"]
    return best_donor, best_amount

print(top_donor(GIFTS))
```

OUTPUT

```
> /Users/you/python-from-zero/programs/ch16_tests/ex7.py(13)top_donor()
-> breakpoint()
(Pdb) p gift["amount"], best_amount, best_donor
(50, 0, '')
(Pdb) n
> /Users/you/python-from-zero/programs/ch16_tests/ex7.py(14)top_donor()
-> if gift["amount"] > best_amount:
(Pdb) n
> /Users/you/python-from-zero/programs/ch16_tests/ex7.py(15)top_donor()
-> best_amount = gift["amount"]
(Pdb) n
> /Users/you/python-from-zero/programs/ch16_tests/ex7.py(12)top_donor()
-> for gift in donations:
(Pdb) p best_amount, best_donor
(50, '')
(Pdb) c
> /Users/you/python-from-zero/programs/ch16_tests/ex7.py(13)top_donor()
-> breakpoint()
(Pdb) n
> /Users/you/python-from-zero/programs/ch16_tests/ex7.py(14)top_donor()
-> if gift["amount"] > best_amount:
(Pdb) n
> /Users/you/python-from-zero/programs/ch16_tests/ex7.py(15)top_donor()
-> best_amount = gift["amount"]
(Pdb) n
> /Users/you/python-from-zero/programs/ch16_tests/ex7.py(12)top_donor()
-> for gift in donations:
(Pdb) p best_amount, best_donor
(100, '')
(Pdb) c
('', 100)
```

Each time a bigger gift is found, `best_amount` changes but `best_donor` stays `''`: the loop never stores the donor's name. The fixed function, with a test and a run on the real file:

```
# programs/ch16_tests/test_ex16_7.py
from donations_report import load_donations

def top_donor(donations):
    """Return (donor, amount) for the single biggest gift."""
    best_donor = ""
    best_amount = 0
    for gift in donations:
        if gift["amount"] > best_amount:
            best_amount = gift["amount"]
            best_donor = gift["donor"]
    return best_donor, best_amount

def test_top_donor():
    gifts = [
        {"date": "2026-01-03", "donor": "Chris Doyle", "amount": 50},
        {"date": "2026-01-10", "donor": "Maple Falls Bakery", "amount": 100},
        {"date": "2026-02-14", "donor": "Grace Kim", "amount": 25},
    ]
    assert top_donor(gifts) == ("Maple Falls Bakery", 100)

if __name__ == "__main__":
    print(top_donor(load_donations("../data/donations.csv")))
```

```
python3 -m pytest -q test_ex16_7.py :
```

OUTPUT

```
. [100%]
1 passed in 0.01s
```

```
python3 test_ex16_7.py :
```

OUTPUT

```
('Maple Falls Rotary Club', 250)
```

Several gifts in the real file are \$250. Because the comparison is `>` (not `>=`), the first one found wins, which is the Rotary Club's gift on January 3.

16.8 Write a smoke test that runs on the real donations file.

```
# programs/ch16_tests/test_smoke.py
from donations_report import load_donations, total, total_by_month

def test_real_donations_file():
    donations = load_donations("../..data/donations.csv")
    assert len(donations) == 130
    assert total(donations) == 7805
    monthly = total_by_month(donations)
    assert sum(monthly.values()) == total(donations)
```

OUTPUT

```
.
1 passed in 0.01s
```

[100%]

The relative path works because pytest runs from `programs/ch16_tests`, two folders below the book's main folder. A smoke test like this would have caught the original bug in the report straight away, since the buggy months added up to only \$500.

Chapter 17

These solutions build on each other, in a copy of the `programs/shelter_manager` folder. New functions go in the module named in each exercise, and new tests in the matching file in `tests/` (add the new names to that file's `import` lines). Tests are run from the `shelter_manager` folder; `-k` tells pytest to run only the tests whose names contain a word, which keeps the output short. The full set of tests still passes at the end. (The outputs were captured with all seven exercises in place, so the "deselected" counts include tests from later exercises.)

17.1 Write `count_by_species(animals)` in `reports.py`, with a test.

```
# added to reports.py
def count_by_species(animals):
    """Return a dict such as {"Dog": 20, "Cat": 16, ...}."""
    counts = {}
    for animal in animals:
        counts[animal.species] = counts.get(animal.species, 0) + 1
    return counts
```

```
# added to tests/test_reports.py
from animals import Animal
from reports import count_by_species

def test_count_by_species():
    animals = [
        Animal("SP-001", "Peanut", "Dog", "Labrador mix", "M", 4.5, 28.6,
              "2026-01-07"),
        Animal("SP-002", "Truffle", "Cat", "Tuxedo", "F", 1, 3.9,
              "2026-01-07"),
        Animal("SP-007", "Gus", "Dog", "Labrador mix", "F", 2, 26.8,
              "2026-01-28"),
    ]
    assert count_by_species(animals) == {"Dog": 2, "Cat": 1}
    assert count_by_species([]) == {}
```

```
python3 -m pytest -q -k count_by_species :
```

```
OUTPUT
```

```
. [100%]
1 passed, 32 deselected in 0.05s
```

Using it on the real shelter (a small script saved in the `shelter_manager` folder):

```

from reports import count_by_species
from storage import start_shelter

shelter, message = start_shelter()
print(count_by_species(shelter.animals))
print(count_by_species(shelter.with_status("Available")))

```

OUTPUT

```

{'Dog': 20, 'Cat': 16, 'Guinea pig': 5, 'Rabbit': 7}
{'Dog': 9, 'Cat': 7, 'Rabbit': 4, 'Guinea pig': 1}

```

This is the counting pattern from Chapter 9. All 48 animals are counted first, then only the 21 available ones.

17.2 Write `family_friendly(shelter)`, the available animals that are good with kids.

```

# added to reports.py
def family_friendly(shelter):
    """The available animals that are good with kids."""
    return [animal for animal in shelter.with_status("Available")
            if animal.good_with_kids]

```

```

# added to tests/test_reports.py
from animals import Animal, Shelter
from reports import family_friendly

def test_family_friendly():
    shelter = Shelter()
    shelter.add(Animal("SP-010", "Scout", "Dog", "Greyhound", "M", 4.5,
                      28.9, "2026-02-03", good_with_kids=True))
    shelter.add(Animal("SP-001", "Peanut", "Dog", "Labrador mix", "M", 4.5,
                      28.6, "2026-01-07", good_with_kids=False))
    shelter.add(Animal("SP-002", "Truffle", "Cat", "Tuxedo", "F", 1, 3.9,
                      "2026-01-07", status="Medical hold",
                      good_with_kids=True))
    assert [a.name for a in family_friendly(shelter)] == ["Scout"]

```

```
python3 -m pytest -q -k family_friendly:
```

OUTPUT

```
. [100%]  
1 passed, 32 deselected in 0.05s
```

```
from reports import animal_table, family_friendly  
from storage import start_shelter  
  
shelter, message = start_shelter()  
friendly = family_friendly(shelter)  
print(animal_table(friendly))  
print(f"{len(friendly)} family-friendly animals are waiting for a home.")
```

OUTPUT

ID	Name	Species	Age	Status
SP-010	Scout	Dog	4.5	Available
SP-014	Kiwi	Dog	3.0	Available
SP-019	Bramble	Rabbit	2.0	Available
SP-020	Ziggy	Rabbit	6.0	Available
SP-023	Sage	Cat	2.0	Available
SP-027	Otis	Rabbit	1.0	Available
SP-028	Milo	Dog	1.3	Available
SP-031	Rusty	Guinea pig	1.0	Available
SP-032	Bear	Cat	3.0	Available
SP-036	Pepper	Cat	8.0	Available
SP-037	Juniper	Dog	10.0	Available
SP-043	Tilly	Cat	11.5	Available
SP-046	Clover	Rabbit	0.5	Available
SP-047	Nugget	Dog	2.0	Available

14 family-friendly animals are waiting for a home.

The test's small shelter includes one animal of each kind that must be left out: one not good with kids, and one good with kids but on medical hold. Reusing `animal_table()` gives the output the same look as the rest of the program.

17.3 Add `Shelter.return_from_foster(animal_id)`, with tests for all three cases.

```
# added to the Shelter class in animals.py
def return_from_foster(self, animal_id):
    """Bring a foster animal back to the shelter as Available."""
    animal = self.find(animal_id)
    if animal is None:
        raise ValueError(f"No animal with ID {animal_id}")
    if animal.status != "Foster":
        raise ValueError(f"{animal.name} is not in foster care")
    if self.spaces_free(animal.species) <= 0:
        raise ValueError(f"No free space for {animal.name}")
    animal.status = "Available"
```

```
# added to tests/test_animals.py
def test_return_from_foster():
    shelter = make_shelter()
    shelter.return_from_foster("SP-009")
    assert shelter.find("SP-009").status == "Available"

def test_only_foster_animals_can_return():
    shelter = make_shelter()
    with pytest.raises(ValueError):
        shelter.return_from_foster("SP-001")    # Peanut is Available

def test_no_return_when_the_kennels_are_full():
    shelter = make_shelter()
    for number in range(19):    # Peanut plus 19 more fill all 20
        shelter.add(Animal(f"SP-{100 + number}", "Extra", "Dog", "Mix",
                           "M", 3, 10, "2026-10-01"))
    with pytest.raises(ValueError):
        shelter.return_from_foster("SP-009")
```

```
python3 -m pytest -q -k foster:
```

OUTPUT

```
..
2 passed, 31 deselected in 0.05s [100%]
```

Only two tests ran: the word "foster" is not in the name

`test_no_return_when_the_kennels_are_full`, so run `python3 -m pytest -q` to run all of them (33 pass). The last test fills the kennels with 19 extra dogs (Peanut is already in one), then checks that Poppy cannot come back. `f"SP-{100 + number}"` gives each extra dog its own ID.

17.4 Add `Shelter.update_weight(animal_id, weight_kg)`, refusing weights of 0 or less.

```
# added to the Shelter class in animals.py
def update_weight(self, animal_id, weight_kg):
    """Record a new weight in kg after a weigh-in."""
    animal = self.find(animal_id)
    if animal is None:
        raise ValueError(f"No animal with ID {animal_id}")
    if weight_kg <= 0:
        raise ValueError("Weight must be more than 0 kg")
    animal.weight_kg = weight_kg
```

```
# added to tests/test_animals.py
def test_update_weight():
    shelter = make_shelter()
    shelter.update_weight("sp-001", 27.9)
    assert shelter.find("SP-001").weight_kg == 27.9
    assert shelter.find("SP-001").daily_food() == 418

def test_update_weight_refuses_zero_and_unknown_ids():
    shelter = make_shelter()
    with pytest.raises(ValueError):
        shelter.update_weight("SP-001", 0)
    with pytest.raises(ValueError):
        shelter.update_weight("SP-999", 5)
```

`python3 -m pytest -q -k weight:`

OUTPUT

```
..
2 passed, 31 deselected in 0.05s
```

[100%]

The first test also checks that the new weight flows through to the food rule: $27.9 \times 15 = 418.5$, and `round()` makes that 418, because Python rounds an exact half to the nearest even number. A test is exactly where such surprises show up.

17.5 Export the available animals to `available.csv` with `csv.DictWriter`.

```
# added to storage.py
def export_available_csv(shelter, folder):
    """Write the available animals to available.csv; return its path."""
    folder.mkdir(exist_ok=True)
    path = folder / "available.csv"
    rows = [animal.to_dict() for animal in shelter.with_status("Available")]
    with open(path, "w", encoding="utf-8", newline="") as file:
        writer = csv.DictWriter(file, fieldnames=list(rows[0].keys()))
        writer.writeheader()
        writer.writerows(rows)
    return path
```

```
# tests/test_storage.py: change the top of the file to
import csv

from storage import (DATA, export_available_csv, load_animals_csv,
                    load_donations_csv, load_shelter, save_shelter,
                    start_shelter)

# and add this test at the bottom
def test_export_available_csv(tmp_path):
    shelter, message = start_shelter(tmp_path / "nothing_saved_yet.json")
    path = export_available_csv(shelter, tmp_path / "output")
    with open(path, encoding="utf-8", newline="") as file:
        rows = list(csv.DictReader(file))
    assert len(rows) == 21
    assert rows[0]["name"] == "Peanut"
```

`python3 -m pytest -q -k "export or fresh"` (this also runs the test from Exercise 7):

OUTPUT

```
.. [100%]  
2 passed, 31 deselected in 0.06s
```

`to_dict()` already gives one dict per animal with the CSV column names as keys, so `DictWriter` can write them directly; `fieldnames` comes from the first row's keys. The test writes into `tmp_path`, never into the real `output/` folder. (If no animals were available, `rows[0]` would fail; a more careful version would check for an empty list first.)

17.6 Record a new donation: a `Shelter` method and a menu option.

```
# added to the Shelter class in animals.py  
def add_donation(self, donor, amount, method, day):  
    """Record a new donation of a whole number of dollars."""  
    if amount <= 0:  
        raise ValueError("A donation must be more than $0")  
    self.donations.append({"date": day.isoformat(), "donor": donor,  
                           "amount": amount, "method": method})
```

```
# added to tests/test_animals.py (which now also imports date)  
def test_add_donation():  
    shelter = make_shelter()  
    shelter.add_donation("Ken Adebayo", 20, "cash", date(2026, 10, 6))  
    assert shelter.donations == [{"date": "2026-10-06",  
                                  "donor": "Ken Adebayo", "amount": 20,  
                                  "method": "cash"}]  
    with pytest.raises(ValueError):  
        shelter.add_donation("Nobody", 0, "cash", date(2026, 10, 6))
```

`python3 -m pytest -q -k donation :`

OUTPUT

```
... [100%]  
3 passed, 30 deselected in 0.05s
```

(The word "donation" also matches two older tests, `test_donations_import` and `test_no_donations`.) In `main.py`, add a function for the new menu choice:

```
def record_donation(shelter, today):
    """Ask for a new donation and record it. Returns True."""
    donor = ask_text("Donor: ")
    amount = ask_int("Amount in whole dollars: ", 1, 100000)
    method = ask_choice("Method (cash, card, online): ",
                        ["cash", "card", "online"])
    shelter.add_donation(donor, amount, method, today)
    print(f"Thank you, {donor}! ${amount:,} recorded.")
    return True
```

Then add 9) Record a donation to the end of the last line of `MENU`, change the menu question to `ask_int("Choose 1-9: ", 1, 9)`, and add a branch to the `if / elif` chain:

```
elif choice == 9:
    if record_donation(shelter, today):
        unsaved = True
```

A session (run on October 6, 2026; we answered `n` when quitting, so nothing was saved):

OUTPUT

Loaded 48 animals and 130 donations from the CSV files (no saved file yet).

=== Sunny Paws Shelter Manager ===

- | | | |
|-----------------|--------------|----------------------|
| 1) List animals | 2) Search | 3) Add an animal |
| 4) Adopt | 5) Donations | 6) Export report |
| 7) Save | 8) Quit | 9) Record a donation |

Choose 1-9: 9

Donor: Ken Adebayo

Amount in whole dollars: twenty

Please enter a whole number from 1 to 100000.

Amount in whole dollars: 20

Method (cash, card, online): CASH

Thank you, Ken Adebayo! \$20 recorded.

=== Sunny Paws Shelter Manager ===

- | | | |
|-----------------|--------------|----------------------|
| 1) List animals | 2) Search | 3) Add an animal |
| 4) Adopt | 5) Donations | 6) Export report |
| 7) Save | 8) Quit | 9) Record a donation |

Choose 1-9: 5

Donations: 131 gifts, total \$7,825, average \$59.73

By month:

Jan 2026	\$	1,855
Feb 2026	\$	1,050
Mar 2026	\$	1,135
Apr 2026	\$	1,425
May 2026	\$	1,410
Jun 2026	\$	930
Oct 2026	\$	20

By method:

card	\$	2,400
cash	\$	2,570
online	\$	2,855

Top donors:

Maple Falls Bakery	\$	2,550
Maple Falls Rotary Club	\$	1,400
Priya Shah	\$	675

```

=== Sunny Paws Shelter Manager ===
1) List animals    2) Search        3) Add an animal
4) Adopt           5) Donations     6) Export report
7) Save            8) Quit          9) Record a donation
Choose 1-9: 8

Save your changes first? (y/n) n
Goodbye from Sunny Paws!

```

`ask_int()` refused "twenty", `ask_choice()` accepted "CASH" and stored it as `cash`, and the summary now has 131 gifts and a new October line.

17.7 Add a `--fresh` command-line option with `argparse`.

In `storage.py`, `start_shelter()` gets a third parameter and only loads the saved file when it is `False`:

```

def start_shelter(save_file=SAVE_FILE, data_folder=DATA, fresh=False):
    """Load the saved shelter, or import the CSV files the first time.

    Returns the shelter and a message saying where the data came from.
    """
    if Path(save_file).exists() and not fresh:
        shelter = load_shelter(save_file)
        source = Path(save_file).name
    else:
        shelter = Shelter()
        for animal in load_animals_csv(Path(data_folder) / "animals.csv"):
            shelter.add(animal)
        shelter.donations = load_donations_csv(
            Path(data_folder) / "donations.csv")
        if fresh:
            source = "the CSV files (saved file ignored)"
        else:
            source = "the CSV files (no saved file yet)"
    message = (f"Loaded {len(shelter.animals)} animals and "
               f"{len(shelter.donations)} donations from {source}.")
    return shelter, message

```

In `main.py`, add `import argparse` at the top and read the option at the start of `main()`:

```
def main():
    parser = argparse.ArgumentParser(
        description="The Sunny Paws Shelter Manager.")
    parser.add_argument("--fresh", action="store_true",
                        help="ignore the saved file and import the CSV "
                        "files again")
    args = parser.parse_args()
    try:
        shelter, message = storage.start_shelter(fresh=args.fresh)
    except (OSError, ValueError) as error:
        print(f"Could not load the shelter's data: {error}")
        return
    print(message)
```

A test for the new parameter, added to `tests/test_storage.py`:

```
def test_fresh_ignores_the_saved_file(tmp_path):
    save_file = tmp_path / "shelter_data.json"
    shelter, message = start_shelter(save_file)
    save_shelter(shelter, save_file)
    shelter, message = start_shelter(save_file, fresh=True)
    assert "CSV" in message
```

`python3 main.py --help`:

```
OUTPUT
usage: main.py [-h] [--fresh]

The Sunny Paws Shelter Manager.

options:
  -h, --help  show this help message and exit
  --fresh     ignore the saved file and import the CSV files again
```

After saving once (so that `shelter_data.json` exists), `python3 main.py --fresh`:

OUTPUT

Loaded 48 animals and 130 donations from the CSV files (saved file ignored).

=== Sunny Paws Shelter Manager ===

1) List animals 2) Search 3) Add an animal
4) Adopt 5) Donations 6) Export report
7) Save 8) Quit 9) Record a donation

Choose 1-9: 8

Goodbye from Sunny Paws!

`action="store_true"` makes `args.fresh` `True` when `--fresh` is typed and `False` otherwise (Chapter 15). Nothing is deleted: if you save in this session, the fresh data replaces the old save file, so the option is a quick way to start over on purpose. With all seven exercises in place, `python3 -m pytest -q` runs 33 tests:

OUTPUT

..... [100%]
33 passed in 0.23s

Chapter 18

The chart solutions run from the `notebooks` folder (as a notebook cell or a script saved there), so the data is at `../data/` and the charts go into `notebooks/output/`. They need `matplotlib` (`python3 -m pip install matplotlib`, or `py -m pip install matplotlib` on Windows).

18.1 Count the animals of each species and draw a bar chart of the counts.

```

import csv
from pathlib import Path

import matplotlib.pyplot as plt

counts = {}
with open("../data/animals.csv", encoding="utf-8", newline="") as file:
    for row in csv.DictReader(file):
        species = row["species"]
        counts[species] = counts.get(species, 0) + 1
print(counts)

plt.figure(figsize=(6, 4))
plt.bar(list(counts.keys()), list(counts.values()), color="#2A9D8F")
plt.title("Sunny Paws animals by species")
plt.ylabel("Number of animals")
Path("output").mkdir(exist_ok=True)
plt.savefig("output/animals_by_species.png", dpi=200)
print("Saved output/animals_by_species.png")

```

OUTPUT

```

{'Dog': 20, 'Cat': 16, 'Guinea pig': 5, 'Rabbit': 7}
Saved output/animals_by_species.png

```

The counting is the dictionary pattern from Chapter 9; matplotlib only draws the result. The species appear in the order they were first met in the file. In a notebook, add `plt.show()` at the end to see the chart under the cell.

18.2 Draw a bar chart of the donations by payment method, with the amounts on the bars.

```

import csv
from pathlib import Path

import matplotlib.pyplot as plt

by_method = {}
with open("../data/donations.csv", encoding="utf-8", newline="") as file:
    for row in csv.DictReader(file):
        method = row["method"]
        by_method[method] = by_method.get(method, 0) + int(row["amount"])
print(by_method)

methods = list(by_method.keys())
amounts = list(by_method.values())
plt.figure(figsize=(6, 4))
bars = plt.bar(methods, amounts, color="#2A9D8F")
plt.bar_label(bars, labels=[f"${amount:," for amount in amounts])
plt.title("Sunny Paws donations by payment method")
plt.ylabel("Dollars")
plt.ylim(0, 3300)
Path("output").mkdir(exist_ok=True)
plt.savefig("output/donations_by_method.png", dpi=200)
print("Saved output/donations_by_method.png")

```

OUTPUT

```

{'online': 2855, 'card': 2400, 'cash': 2550}
Saved output/donations_by_method.png

```

Online giving is the largest of the three. `plt.ylim(0, 3300)` leaves room above the tallest bar for its label. The three totals add up to \$7,805, as they should.

18.3 Read about `statistics.median()` with `help()`, then find the median weight of the dogs.

```

import statistics

help(statistics.median)

```

OUTPUT

Help on function median in module statistics:

median(data)

Return the median (middle value) of numeric data.

When the number of data points is odd, return the middle data point.

When the number of data points is even, the median is interpolated by taking the average of the two middle values:

```
>>> median([1, 3, 5])
```

```
3
```

```
>>> median([1, 3, 5, 7])
```

```
4.0
```

```
import csv
import statistics

dog_weights = []
with open("../data/animals.csv", encoding="utf-8", newline="") as file:
    for row in csv.DictReader(file):
        if row["species"] == "Dog":
            dog_weights.append(float(row["weight_kg"]))

print(len(dog_weights), "dogs")
print("Median weight:", statistics.median(dog_weights), "kg")
print("Mean weight:", round(statistics.mean(dog_weights), 1), "kg")
```

OUTPUT

20 dogs

Median weight: 13.35 kg

Mean weight: 17.3 kg

There are 20 dogs, an even number, so the median is the average of the two middle weights, exactly as the documentation says. The mean is higher than the median because a few big dogs (around 30 kg) pull the average up, while small dogs like Pickles and Coco barely pull it down.

18.4 Write a minimal example that reproduces the `TypeError`, then fix it.

```
# programs/minimal.py
age = 3
print("Age: " + age + " years")
```

OUTPUT

```
Traceback (most recent call last):
  File "/Users/you/python-from-zero/programs/minimal.py", line 2, in <module>
    print("Age: " + age + " years")
                ~~~~~^~~~~
TypeError: can only concatenate str (not "int") to str
```

Two lines are enough: `+` cannot join text and a number (Chapter 3). In a question you would paste these two lines, the full error and one sentence such as "I expected `Age: 3 years` ." The fix is to turn the number into text, or, better, to use an f-string:

```
age = 3
print("Age: " + str(age) + " years")
print(f"Age: {age} years")
```

OUTPUT

```
Age: 3 years
Age: 3 years
```

18.5 Put a project under Git, commit, change one thing and commit again.

Here we used the small project from Exercise 7, in `programs/rota/` (any folder works). First a `.gitignore` :

```
__pycache__/
.pytest_cache/
.venv/
```

Then, in a terminal in that folder:

```
git init -b main
git add .
git commit -m "Who is on today? First version with tests"
```

OUTPUT

```
[main (root-commit) e5d8b8f] Who is on today? First version with tests
3 files changed, 64 insertions(+)
create mode 100644 .gitignore
create mode 100644 rota.py
create mode 100644 test_rota.py
```

(`git init` also printed its one-line `Initialized empty Git repository` message.) Next we changed the "Nobody is on the rota that day." message in `rota.py` to add "Ask Ken!", and committed again:

```
git status --short
```

OUTPUT

```
M rota.py
```

```
git add rota.py
git commit -m "Suggest asking Ken when nobody is on"
git log --oneline
```

OUTPUT

```
[main 8b15878] Suggest asking Ken when nobody is on
1 file changed, 1 insertion(+), 1 deletion(-)
8b15878 Suggest asking Ken when nobody is on
e5d8b8f Who is on today? First version with tests
```

Your commit IDs will be different. "1 insertion(+), 1 deletion(-)" is how Git describes a changed line: the old version removed and the new one added.

18.6 Create and activate a virtual environment, and show that `sys.prefix` points inside it.

```
cd programs/rota
python3 -m venv .venv          # py -m venv .venv on Windows
source .venv/bin/activate      # .venv\Scripts\activate on Windows
python -c "import sys; print(sys.prefix)"
```

OUTPUT

```
/Users/you/python-from-zero/programs/rota/.venv
```

`python -m pip freeze` in a brand-new environment prints nothing at all, because nothing has been installed in it yet: proof that it is separate from your main Python, where `pytest` and others are installed. After `python -m pip install pytest`, `python -m pip freeze > requirements.txt` writes a file with one line per package, such as `pytest==` followed by the version number, plus a few small packages that `pytest` itself needs. Anyone can then recreate the same environment with `python -m pip install -r requirements.txt`. Type `deactivate` to leave the environment.

18.7 Plan and start a small project: a "Who is on today?" helper, with requirements, a function and tests.

```

# programs/rota/rota.py
"""Who is on today? A small helper for Sunny Paws.

Requirements:
- Read the volunteers from data/volunteers.json.
- Given a day such as "Sat", list the volunteers working that day,
  with their skills.
- Accept the day in any capitals ("sat", "SAT").
- Say so politely on Mondays, when the shelter is closed.
"""

import json
from pathlib import Path

DATA = Path(__file__).parent.parent.parent / "data"

def load_volunteers(path):
    with open(path, encoding="utf-8") as file:
        return json.load(file)["volunteers"]

def volunteers_on(volunteers, day):
    """Return the volunteers who work on day ("Tue", "Wed", ...)."""
    day = day.strip().title()
    return [person for person in volunteers if day in person["days"]]

def main():
    volunteers = load_volunteers(DATA / "volunteers.json")
    day = input("Which day? (Tue to Sun) ")
    if day.strip().title() == "Mon":
        print("The shelter is closed on Mondays.")
        return
    working = volunteers_on(volunteers, day)
    if not working:
        print("Nobody is on the rota that day.")
    for person in working:
        print(f"{person['name']:<16}{'', '.join(person['skills'])}")

```

```
if __name__ == "__main__":  
    main()
```

```
# programs/rota/test_rota.py  
from rota import volunteers_on  
  
TEAM = [  
    {"name": "Ken Adebayo", "days": ["Tue", "Wed", "Sat"]},  
    {"name": "Lucy Fairbanks", "days": ["Sat", "Sun"]},  
    {"name": "Ben Carter", "days": ["Sun"]},  
]  
  
def test_saturday():  
    names = [person["name"] for person in volunteers_on(TEAM, "Sat")]  
    assert names == ["Ken Adebayo", "Lucy Fairbanks"]  
  
def test_any_capitals():  
    assert len(volunteers_on(TEAM, " sun ")) == 2  
  
def test_nobody_on_monday():  
    assert volunteers_on(TEAM, "Mon") == []
```

```
python3 -m pytest -q in programs/rota :
```

```
OUTPUT
```

```
... [100%]  
3 passed in 0.01s
```

Two runs of `python3 rota.py` :

OUTPUT

```
Which day? (Tue to Sun) sat
Ken Adebayo      dog walking, training
Lucy Fairbanks   cat care, photography
Tomas Novak      small animals, cleaning
Aisha Bello      cat care, fundraising
```

OUTPUT

```
Which day? (Tue to Sun) Mon
The shelter is closed on Mondays.
```

The requirements, written first as a docstring, decided the design: one function that does the real work (and is easy to test with a tiny hand-made team), plus a `main()` that talks to the user. The data path is built from `Path(__file__)`, as in Chapter 17, so the program works from any folder. Next steps for this project could be a test for `load_volunteers()` with the real file, and an option to pick the day automatically with `date.today()`.